

智能体时代的自动化渗透测试：进展与展望

王鹏飞¹⁾ 张文华¹⁾ 王恩泽¹⁾ 周源¹⁾ 陈安莹²⁾
夏天¹⁾ 陶仪¹⁾ 解炜¹⁾

¹⁾(国防科技大学计算机学院, 长沙 410073)

²⁾(北京云起无垠科技有限公司, 北京 100010)

摘 要

渗透测试是通过授权的模拟攻击行为对计算机系统、网络或应用的安全性进行评估, 是发现潜在脆弱性、验证安全防护有效性的重要手段。传统自动化渗透测试方法仍较为依赖人工经验, 在复杂、动态和大规模网络环境中的适应性与自主决策能力有限。近年来, 大语言模型 (Large Language Model, LLM) 和多智能体 (Multi-Agent) 技术快速发展, 推动自动化渗透测试研究由规则驱动和单工具调用, 逐步向基于大模型和智能体的智能化协同测试演进。本文围绕智能体时代自动化渗透测试的研究进展, 系统梳理了近 3 年的 44 项代表性工作, 重点从多智能体协作、领域知识增强、短期记忆管理、推理能力提升、工具调用优化以及测试评估等方面总结该方向的关键技术路径与演进脉络; 在此基础上, 分析当前研究面临的主要挑战, 并对未来发展趋势进行了展望。本文可为智能化渗透测试技术研究与设计提供参考。

关键词 渗透测试; 大语言模型; 多智能体协同; 智能化渗透测试; 渗透测试测评

中图法分类号 TP393.08

DOI 号: * 投稿时不提供 DOI 号

Automated Penetration Testing in the Era of AI Agents: Progress and Prospects

WANG Peng-Fei¹⁾ ZHANG Wen-Hua¹⁾ WANG En-Ze¹⁾ ZHOU Yuan¹⁾ CHEN An-Ying²⁾ XIA Tian¹⁾
TAO Yi¹⁾ XIE Wei¹⁾

¹⁾(College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China)

²⁾Clouditera, Beijing 10010)

Abstract

Penetration testing is an important technique for evaluating the security of computer systems, networks, and applications by simulating adversarial behaviors under authorization. It plays a vital role in identifying potential vulnerabilities and validating the effectiveness of security defenses. Traditional automated penetration testing methods still rely heavily on human expertise while exhibiting limited adaptability and autonomous decision-making capability in complex, dynamic, and large-scale network environments. In recent years, large language models (LLMs) and multi-agent techniques have advanced rapidly, driving automated penetration testing from rule-based and single-tool invocation paradigms toward intelligent, collaborative testing frameworks. Focusing on the research progress of automated penetration testing in the era of AI agents, this paper systematically reviews 44 representative studies published over the past three years. It summarizes the key technical paths and evolutionary trends of this field from the perspectives of multi-agent collaboration, domain knowledge enhancement, short-term memory management, reasoning capability improvement, tool invocation optimization, and evaluation methodologies. On this basis, the paper further analyzes the major challenges in current research and discusses possible future directions. This survey is expected to provide useful references for future research and system design in intelligent penetration testing.

Keywords Penetration testing; Large language models; Multi-agent collaboration; Intelligent pentest; Pentest evaluation

收稿日期: - - ; 最终修改稿收到日期: - - . * 投稿时不填写此项 *.

本课题得到国家自然科学基金 (No.62272472)、湖南省科技创新计划 (No.2024RC3136) 资助。王鹏飞 (通信作者), 男, 1988 年生, 博士, 副研究员, 计算机学会 (CCF) 会员 (D8405S), 主要研究领域为软件漏洞挖掘、渗透测试。E-mail: pfwang@nudt.edu.cn. 第 1 作者手机号码: 15526429185, E-mail: pfwang@nudt.edu.cn

1 引言

渗透测试是在合法授权前提下,通过模拟攻击者行为对目标系统安全性进行评估的一类主动安全测试技术。作为“以攻促防”的典型实践,渗透测试能够帮助组织提前识别网络、主机、应用及业务流程中的安全薄弱环节,为漏洞修复、风险评估与安全加固提供依据,因此在网络安全防御体系中具有重要作用。

传统渗透测试通常由人工主导完成。测试人员需要结合公开情报搜集、资产测绘、漏洞分析、利用验证以及后渗透等多个阶段,持续进行环境理解、攻击路径判断和工具选择。尽管 Nmap、Nessus、Metasploit 等工具已经在资产发现、漏洞扫描和利用执行等环节提供了较强支撑,但整个测试流程在目标理解、结果研判、路径规划和多步任务衔接等方面仍然较大程度依赖人员经验。这种模式在面对大规模、异构化、动态变化的网络环境时,存在测试成本较高、过程耗时较长、结果稳定性受人员能力影响较大等问题,难以满足复杂场景下对高效率与高适应性的安全评估需求。

为降低人工参与成本、提升测试效率,研究者不断探索自动化渗透测试技术。已有工作主要围绕基于规则的任务编排、基于知识库的漏洞匹配以及基于强化学习的攻击路径搜索等方向展开。特别是强化学习方法通过将渗透测试过程形式化为智能体与环境交互的序贯决策问题,为自动化渗透测试提供了研究路径。然而,该类方法通常依赖较强的环境建模与状态抽象,在复杂真实网络环境中容易受到状态空间庞大、奖励稀疏、训练成本高以及策略迁移性不足等因素制约,其在开放环境中的泛化能力和可解释性仍有待提升。

近年来,大模型的快速发展为自动化渗透测试研究带来了新的技术机遇。一方面,大模型在自然语言理解、知识调用、上下文推理和任务规划等方面表现出较强能力,使其能够较好地承担测试过程中的信息总结、攻击面分析、攻击路径推断和命令生成等任务;另一方面,多智能体技术通过角色分工、协同交互和状态传递,为复杂测试任务的分阶段执行与模块化组织提供了新的实现方式。研究者开始将大模型、多智能体协作、检索增强生成、外部工具调用与记忆机制结合,构建具备一定自主分析与执行能力的智能化渗透测试系统。

基于上述背景,本文聚焦智能体时代自动化渗透测试的研究进展,系统梳理近年来基于大语言模型和多智能体技术的代表性研究工作,重点分析该方向在多智能体协作、领域知识增强、短期记忆管理、推理与规划能力提升、工具调用优化以及评测

方法等方面的关键进展,总结现有研究的共性思路、主要挑战与发展趋势,以期后续研究提供较为系统的参考。本文的主要工作概括如下:

- 系统梳理了近 3 年基于大语言模型和智能体技术的自动化渗透测试代表性研究工作;
- 从多智能体协作、领域知识增强、短期记忆管理、推理与规划、工具调用和评测方法六个方面总结了关键技术路径与演进脉络;
- 分析了当前研究在真实环境适应性、经验复用、评测基准等方面面临的主要挑战;
- 结合该方向的发展现状,对未来值得关注的研究方向进行了展望。

2 渗透测试介绍

2.1 渗透测试基本原理

渗透测试是一项“以攻促防”的主动防御实践。一次规范的传统渗透测试,通常遵循一个系统化、阶段化的生命周期。其标准流程如图 1 所示,主要包含以下五个关键步骤:

(1) **前期交互与信息收集(侦察)**: 此阶段主要工作是尽可能全面地搜集关于目标的一切信息。测试人员会使用公开资源(OSINT)和扫描工具(如 Nmap)来发现网络资产、域名、IP 地址、开放端口、运行服务及版本等信息,绘制出详细的“攻击面地图”,为后续行动奠定基础。

(2) **威胁建模与漏洞分析**: 基于收集到的信息,测试人员会分析系统架构,识别潜在的攻击入口和高价值目标。随后,结合自动化漏洞扫描工具(如 Nessus)的结果与手动验证,深入分析所发现漏洞的成因、可利用性及可能造成的业务影响。

(3) **漏洞利用与权限提升**: 此阶段是模拟攻击的核心。测试人员会尝试利用已确认的漏洞(例如,使用 Metasploit 框架)来获取对系统的初始访问权限。一旦进入系统内部,便会进一步利用系统配置错误或权限漏洞,尝试将访问权限从普通用户提升至管理员级别。

(4) **后渗透与横向移动**: 模拟高级持续性威胁,测试人员在取得权限后,会尝试在系统中建立持久化访问通道(如创建后门账户),并以此为跳板,在网络内部进行横向移动,探查并控制其他关键服务器或数据,以评估安全边界的真实有效性。

(5) **报告编制与结果分析**: 最后一个阶段是梳理与呈现。测试人员需详细记录所有测试步骤、利用成功的漏洞、获取的敏感数据以及整个攻击路径。最终输出一份结构化的报告,清晰说明发现的安全

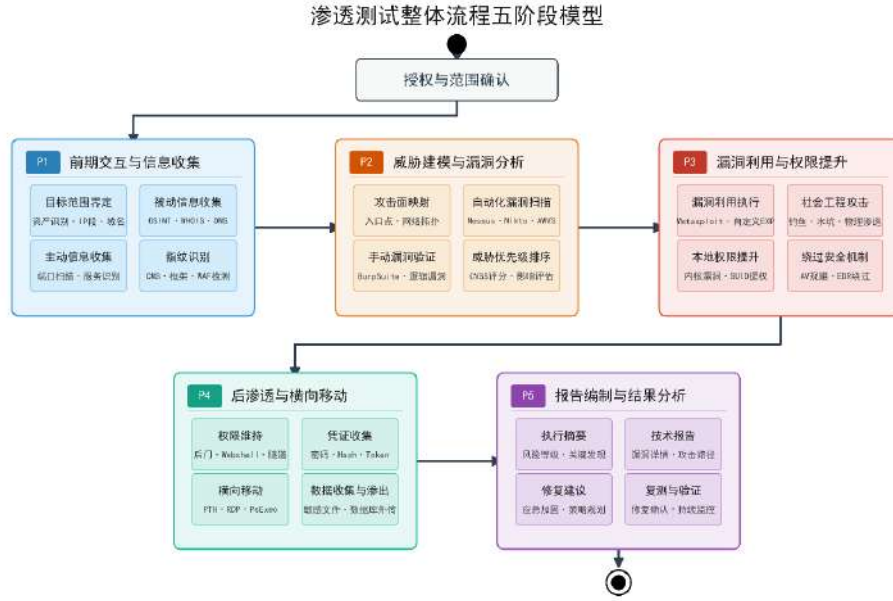


图1 渗透测试整体流程图

问题、风险等级，并提供具体、可操作的技术与管理修复建议。

2.2 基于强化学习的渗透测试

传统渗透测试高度依赖专业人员的经验与技能，存在效率低下、成本高昂、覆盖范围有限等不足，难以适配现代网络系统和基础设施的复杂性与规模化需求。强化学习（RL）凭借其通过与环境交互、试错学习最优策略的核心特性，成为自动化渗透测试领域的研究热点。2019年，Schwartz等人首次提出在模拟环境里用DQN做渗透测试决策[1]，随后，国内外研究者围绕强化学习在渗透测试中的应用开展了大量探索[2-7]，取得了一系列突破性成果，同时也面临着诸多亟待解决的挑战。

基于强化学习的渗透测试研究聚焦于三大方向：在算法优化层面，研究者主要通过改进强化学习及相关衍生算法，缓解渗透测试中的效率、稳定性及稀疏奖励等核心问题，提升模型训练与测试的整体性能[8-10]；在场景适配层面，重点突破复杂网络环境的适配难题，强化智能体的自主决策与多步攻击推理能力，提升策略在真实场景中的鲁棒性与可迁移性[11]；在性能提升层面，通过融合多学科技术、优化路径搜索与样本利用方式，实现漏洞发现效率与测试覆盖面的双重提升[12]。整体而言，当前基于强化学习的渗透测试研究已逐步推动自动化渗透测试向高效化、智能化方向发展。

尽管强化学习为渗透测试的智能化提供了可行路径，但在实际应用中仍面临诸多核心挑战。首先，

在环境建模与状态感知方面，受限于算法本身的状态表征能力，难以处理真实网络的动态性、异构性与不确定性，且多依赖仿真场景建模，导致训练策略的迁移性较差，马尔可夫性假设易失效；其次，在样本利用与决策可解释性上，存在稀疏奖励问题，有效漏洞利用的正奖励事件极少，高质量多步攻击样本稀缺、获取成本高，同时其“黑箱”特性导致决策过程难以追溯，缺乏可解释性与风险控制能力。最后，在大规模场景中适配能力差，随着网络规模扩大，动作空间急剧增加，易出现测试效率大幅下降，难以满足大规模网络的时效性需求。

2.3 基于大模型微调或训练的渗透测试

近年来，随着大模型在语义理解、人机交互、复杂推理等方面能力的快速提升，研究者开始尝试通过领域微调的方式，将大模型用于渗透测试任务优化，其核心在于通过领域适配和高质量轨迹学习，将大模型从“通用语言理解器”转变为“具备渗透测试知识的专业助手”。基于微调或训练的大模型方法能够直接继承通用大模型在代码、命令、漏洞描述和自然语言交互上的先验知识，再通过网络安全领域数据进一步适配，因此在复杂渗透语义建模、任务泛化、命令生成和多步推理等方面通常优于基于强化学习的自动化渗透测试技术路线。从现有研究来看，该方向大致可分为两类：

第一类是面向特定渗透子任务的专用微调模型。典型代表是CIPHER[13]，该工作将大模型在专门构建的渗透测试指导数据（含靶机实战、攻击

技术、开源工具文档、渗透测试 Write-up 等) 上进行微调, 使大模型以聊天助手形式提供更具针对性的攻击思路、漏洞利用建议和命令辅助指导; 类似地, RedShell[14] 聚焦于 Windows 环境下的攻击性 PowerShell 生成任务, 通过 CTF walkthrough、恶意 PowerShell 数据集、GitHub 等来源的样本结合 LoRA/Unsloth 等轻量化微调技术, 提升大模型的 PowerShell 生成能力, 帮助渗透测试人员完成攻击脚本编写。

第二类是基于轨迹监督或离线合成交互数据的训练方法。代表性工作是 Cyber-Zero[15], 利用公开 CTF write-up 和人物角色驱动的模拟机制, 重建智能体与环境之间的多轮交互轨迹, 实现在无需真实运行时环境条件下训练出具备较强安全任务能力模型的框架; 类似的, CTF-Dojo[16] 是一个网络安全大模型训练环境, 通过 CTF-Forge 框架自动将公开资源转化为可运行的 Docker 化 CTF 挑战, 用于训练能在 CTF 任务中寻找漏洞的语言模型, 使大模型能够从真实执行反馈中学习漏洞挖掘与利用行为, 减少传统人工配置成本。

综上, 基于微调或训练的大模型渗透测试方法相较于基于强化学习的技术路线具有三方面优势。一是大模型能够充分利用预训练阶段获得的通用知识, 在面对漏洞描述、脚本语义和系统交互等复杂输入时表现出更强的语义理解与推理能力; 二是这类方法更容易吸收真实渗透测试知识, 包括 write-up、命令序列、脚本样本和专家操作轨迹; 三是大模型方法能够通过监督微调或离线训练减少对高成本在线探索的依赖, 从而缓解强化学习方法中普遍存在的样本效率低和环境构建困难问题。

然而, 这一路线仍存在不足。首先, 其性能高度依赖训练数据质量与覆盖范围, 无论是渗透测试知识问答、恶意脚本样本, 还是 CTF 轨迹数据, 都可能存在任务类型单一、真实环境覆盖不足以及数据分布偏窄的问题。其次, 许多方法虽然在 benchmark 或 CTF 任务上表现突出, 但面对真实企业网络、复杂内网横向移动、多资产动态交互等场景时, 泛化能力仍有待验证。更重要的是, 当前多数基于微调或训练的工作仍主要依赖单一模型进行决策, 这使其在任务分工、并行侦察、规划与执行解耦、失败恢复、自我校验以及跨工具协同等方面存在天然短板。

3 智能体时代渗透测试的研究进展

近年来, 随着大语言模型推理能力、工具调用能力和任务规划能力的持续提升, 自动化渗透测试研究逐步由早期的单轮辅助式使用, 发展为围绕复杂测试流程开展的多阶段、模块化与协同化系统设

计。与传统依赖预定义规则或强化学习策略的自动化方法相比, 基于大语言模型和智能体的渗透测试系统在环境理解、知识迁移、跨阶段信息整合以及复杂任务编排等方面表现出更强的灵活性, 也因此成为当前自动化渗透测试研究的重要方向。本节围绕该方向的代表性工作, 对其技术路线与演进特征进行归纳分析。

3.1 文献调研范围与筛选方法

为较为系统地梳理智能体时代自动化渗透测试的研究进展, 本文对 2023 年 8 月至 2026 年 4 月期间公开发表或发布于预印本平台的相关研究进行了检索与筛选。检索渠道主要包括 Google Scholar、arXiv、GitHub 以及部分安全和软件工程领域学术会议、期刊的公开资源。检索关键词包括 “pentest”、“penetration testing”、“LLM pentest”、“multi-agent pentest”、“AI-powered pentest” 等英文关键词及其对应中文表达, 并结合具体系统名称进行补充追踪。在筛选过程中, 本文重点保留同时满足以下条件的研究工作: (1) 研究主题与自动化渗透测试流程直接相关; (2) 方法设计中明确引入大模型、智能体或多智能体协作机制; (3) 论文对系统架构、关键方法或实验评估进行了相对完整的描述。

对于仅涉及通用安全问答、单点漏洞检测、纯防御分析与渗透测试流程关联较弱的工作, 本文未纳入核心分析范围。经人工筛选与去重后, 最终保留 44 项具有代表性的研究工作作为重点分析对象, 如表 1 所示。由于该方向发展迅速, 部分工作仍处于预印本、开源迭代或 workshop 发表阶段, 其题名、版本和发表信息可能随时间发生更新。

表 1: 基于大模型与多智能体的自动化渗透测试代表性工作对比

序号	工具（框架）名称	目标阶段	智能体形态	知识增强	记忆管理	规划推理	工具调用	结果验证	评测场景	开源信息
1	PentestGPT [17] (arXiv 2023, USENIX Security 2024)	全流程	单智能体/模块化	文中未强调外部知识增强	基于渗透测试任务树 (PTT) 维护状态, 按测试步骤记录上下文与阶段状态	“推理-生成-解析”三模块协同决策	多工具链自主调用	○	文中未明确具体 benchmark 名称, 主要为渗透测试任务场景	https://github.com/GreyDGL/PentestGPT
2	PentestAgent[18] (AsiaCCS 2025)	全流程	多智能体	RAG 检索增强 (外部知识库)	RAG + 数据库存储环境信息与攻击知识 (分层树结构)	“侦察-搜索-规划-执行”四智能体协同规划	多工具协同	○	VulHub (67 目标) + HackTheBox (11 挑战)	https://github.com/nbshenxm/pentest-agent
3	HackSynth[19] (arXiv 2024)	全流程	双模块/分层式	—	通过截断长输出, 保留最近关键观察结果	“规划者-总结者”分层决策	命令执行链	—	PicoCTF + OverTheWire (200 dynamic flag tasks)	https://github.com/aielte-research/HackSynth
4	AutoPT[20] (arXiv 2024)	全流程	分层式/流水线	—	基于渗透测试状态机 (PSM) 进行阶段间状态传递, 维护侦察、推理、载荷与利用记录	“侦察-漏洞推理-载荷生成-利用尝试”四阶段流水线推理	漏洞利用工具链	✓	文中未明确具体测试集名称, 主要为 Web penetration testing tasks	https://github.com/securityscience/secsai-autopt
5	PTGroup[21] (ICIC 2024)	全流程	多智能体	通过多提示链增强领域决策	通过“关键信息列表” (Assistant 日志) 回传前序结果, 减少多步测试中的信息遗忘	多提示链顺序切换 + “主控-执行-修正-协助”四智能体协同决策	多工具协同	—	Metasploitable2 + Vulhub + Windows 虚拟机漏洞场景	—
6	VulnBot[22] (arXiv 2025)	全流程	多智能体	RAG 检索外部知识 (HackTricks 等)	”阶段总结 + 记忆检索”保留失败原因、成功步骤与漏洞上下文, 支持跨阶段复用	基于渗透任务图 (PTG) 的任务规划与自纠正	多工具协同	○	AutoPenBench + AI-Pentest-Benchmark	https://github.com/KHenryAegis/VulnBot
7	AutoAttacker[23] (arXiv 2024)	后渗透	单智能体/记忆增强	RAG 检索成功动作经验, 复用历史后渗透知识	通过经验管理器缓存成功动作与阶段结果, 为后续步骤提供可复用经验	后渗透任务导向决策	后渗透工具链/ATT&CK 相关工具	—	MITRE ATT&CK 14 任务 benchmark	https://github.com/binarybrain-009/AUTOATTACKER/tree/main
8	AWE[24] (LAST-X 2026)	漏洞发现与验证	多智能体	结合结构化漏洞分析与环境知识; 持久记忆 (短期 + 长期)	统一状态模型管理输入点、认证状态、载荷尝试记录	上下文接力 + 分层协同推理	浏览器验证引擎 + 多工具协同	✓	XBOW benchmark (104 挑战) + DVWA	https://github.com/stuxlabs/AWE
9	CHAP[25] (LAST-X 2026)	全流程	单智能体轮换 (上下文接力)	—	通过紧凑协议 (handoff protocol) 在不同智能体实例间传递攻击路径、已获权限、未利用漏洞等核心上下文	基于手写协议的上下文接力	多工具协同 (任意 shell 命令)	—	AutoPenBench (11 个挑战, 增强版)	https://github.com/marvang/chap

续下页注:

序号	工具（框架）名称	目标阶段	智能体形态	知识增强	记忆管理	规划推理	工具调用	结果验证	评测场景	开源信息
10	PenForge[26] (ICSE 2026)	漏洞发现与验证	多智能体/动态生成	检索外部安全知识 (Perplexity API)，结合端点扫描与网页分析	结构化动作日志记录 尝试历史，供元规划 器参考	元规划 (Meta-Planner) + 动态专家代理构建	终端扫描 + 网页解析 + 知识检索	✓	CVE-Bench (40 个 CVE)	https://github.com/rohit-1006/Penforge
11	vEcho[27] (arXiv 2026)	漏洞主动发现	多智能体协作	开发者工具套件 (代码导航、项目理解、网络搜索) + 认知记忆模块	认知记忆模块 (临时 + 永久知识库) 维护 临时观察、长期漏洞 线索与传播结果	回响式漏洞传播推理 (EVP)	代码导航 + 项目理解 + 网络搜索	—	CWE-Bench-Java (120 个 Java 项目)	将开源 (论文提及)
12	PTFusion[28] (Information Fusion 2026)	全流程	分层式/半去中心化	动态知识图谱 + 上下文感知 知识融合	短期执行记录 (action history) + 动态知识 图谱关联结果	“主控”战略规划 + “侦察/攻击”战术执行 (两阶段任务规划)	MCP 协议调用多工具 (nmap、dirb、msfconsole、hydra、sqlmap 等)	✓	VulnHub (6 个环境)	—
13	PentestMCP[29] (预印本 Research Square 2025)	全流程	多智能体	RAG 检索增强 (all-MiniLM-L6-v2 向量匹配)	PTG (渗透任务图) + 任务状态管理 (成功/失败/未执行)	三阶段分解 + 任务驱动机制 + 双路径策略 (Plan A/B)	MCP 封装 17 种工具 (nmap、sqlmap、metasploit 等) + 任意命令执行	—	VulHub + NVD (100+ 漏洞，覆盖易/中/难三级)	—
14	Pentest-Chain[30] (电信科学 2025,41(09):119-132)	全流程	多智能体	双路 RAG (外部知识库 Faiss + 内部经验库 Milvus)	内部经验管理器记录 历史经验，结合阶段 摘要实现上下文延续， 支持跨任务经验联想	“环境探针-分 析器-决策器-执 行器-报告器” 五智能体链式 规划与任务分解	多工具协同	—	130 个自建任务 (基于 Vulhub 及自行搭建环境)，覆盖信息收集、漏洞利用、权限维持/提升三类典型渗透测试场景	—
15	AutoPentester[31] (TrustCom 2025)	全流程	分层式	RAG 检索成功 案例知识库 + HackTricks 命令库	复用历史成功渗透测试任务树 (PTT)，保留前序阶段关键上下文	面向 “IP to Shell” 的任务规划	多工具链协同	✓	企业级渗透任务场景	https://github.com/YasodGinige/AutoPentester
16	AutoPentest[32] (arXiv 2025)	全流程	分层式/多工作者	RAG 检索外部知识 (OWASP、CWE、HackTricks 等)	主管与专业化工作者 共享状态，维护任务 上下文	规划者-主管-专业化工作者 分层协作	浏览器自动化 (Playwright) + Shell + Python 执行环境 + NIST NVD 工具	—	HackTheBox (3 台机器: Devvortex、Broker、Codify)	https://github.com/JuliusHenke/autopentest
17	RapidPen[33] (arXiv 2025)	全流程 (IP-to-Shell)	分层式	RAG 检索成功 案例知识库与命令库	使用外部文件存储攻 击上下文，减少长链 交互中的上下文丢失	“高层规划者 + 底层执行者” 分层推理	动态工具替换与多工具协同	✓	Hack The Box Legacy + Blue (MS17-010)	https://github.com/AgenticSec/RapidPen-Edge-Installer

续下页注:

表 1 续表

序号	工具（框架）名称	目标阶段	智能体形态	知识增强	记忆管理	规划推理	工具调用	结果验证	评测场景	开源信息
18	MAPTA[34] (arXiv 2025)	全流程	多智能体	文中未强调外部知识增强	共享 Docker 容器维护攻击状态，协调器与沙箱代理上下文隔离	协调器 + 沙箱代理 + 验证代理三层协作	多工具协同 (run_command、run_python、sandbox_agent 等)	✓	XBOW benchmark (104 挑战) + 10 个开源应用	https://github.com/arthurgervais/mapta
19	Cochise[35] (TOSEM 2025)	全流程/企业内网	多智能体/声明式任务	—	通过上下文服务与资产管理服务持续维护多主机、多跳渗透状态	高层声明式任务规划 + 领域任务智能体执行	企业内网工具链/AD (Active Directory) 相关工具	—	Active Directory 环境	https://github.com/andreashappe/cochise
20	ARACNE[36] (arXiv 2025)	Shell 层工具调用与交互利用	模块化多模型	—	上下文文件存储 + 可选摘要器压缩	规划者-解释者-可选总结器模块化规划	Shell 工具自动调用与结果解析	—	ShellLM + OverTheWire Bandit (33 挑战)	—
21	PentestGPTv2[37] (arXiv 2026)	全流程	单智能体/分层混合	RAG 知识库 (工具文档、CVE、攻击剧本)	状态存储器 (State Store) 维护结构化状态，TDI 跟踪任务难度与证据链	TDA + EGATS (证据引导攻击树搜索)	集成 38 种工具 (nmap、sqlmap、metasploit、bloodhound 等)	—	XBOW + PentestGPT Benchmark (13 机器) + GOAD (5 主机) + HTB Season 8	—
22	TermiAgent[38] (arXiv 2025)	全流程	分层式	自建武器库 (1378 Dockerized exploits + 1077 Metasploit 模块)	渗透记忆树 (PMT) + 定位记忆激活 (LMA) 组织上下文	推理器高层规划 + 协助器低层指令生成 (PMT 引导)	自建武器库 + Metasploit 等	✓	TermiBench (510 hosts, 30 CVEs)	—
23	xOffense[39] (arXiv 2025)	全流程	多智能体/知识增强	微调中等规模开源模型 (Qwen3-32B) 内化渗透测试 CoT 与进攻知识	灰盒阶段提示 + 反思/重规划机制维护任务状态	反思重规划 + 任务协调图 (TCG)	多工具协同	—	AutoPenBench + AI-Pentest-Benchmark	—
24	CHECKMATE[40] (arXiv 2025)	全流程/结构化规划	分层式	形式化规划知识	—	PEP 架构 (规划者-执行者-感受器) + 经典规划增强	集成 14,000+ Metasploit 模块、NSE 脚本、Nuclei 模板等预定义动作	—	Vulnhub (120 个容器)	—
25	RedTeamLLM[41] (arXiv 2025)	全流程/真实红队闭环	单智能体 (多组件)	长期经验沉淀 (Memory Manager 存储任务树)	通过长期记忆沉淀“发现-规划-执行-复现”全过程，支持失败后继续修正	动态计划修正 (Plan Corrector) + 攻击树调整 (ADaPT Enhanced)	多工具协同 (完全终端访问)	—	VulnHub (5 个 VM: Sar、CewiKid、Victim1、WestWild、CTF4)	https://github.com/lre-security-systems-team/redteamllm

续下页注:

表 1 续表

序号	工具（框架）名称	目标阶段	智能体形态	知识增强	记忆管理	规划推理	工具调用	结果验证	评测场景	开源信息
26	Breachseek[42] (arXiv 2024)	全流程	多智能体	文中提及结合 LangGraph 分发任务，知识增强方式未详细展开	通过图节点状态与监督者/评估者共享上下文，避免上下文过载	监督者-渗透测试者-评估者-记录者四智能体协同	多工具分工调用	—	Metasploitable 2	https://github.com/snow10100/pena/
27	Pentest-ai[43] (CSR 2024)	全流程/企业网络	多智能体	MITRE ATT&CK 知识框架增强	通过工作平面、数据平面、控制平面分层共享状态与攻击进度	基于 ATT&CK 战术分层规划	按 ATT&CK 战术分配工具调用	—	MITRE ATT&CK 引导的企业网络任务	—
28	RefPentester[44] (PST 2025, arXiv 2025)	全流程	单智能体 (人机协作)	RAG 检索 MITRE ATT&CK + OWASP Testing Guide 知识	七状态状态机 (PT Stage Machine) + 成功/失败日志管理流程	自反思考 (Reflector) + 状态机规划	多工具协同 (通过命令指导)	✓	Hack The Box Sau 机器	—
29	PenHeal[45] (arXiv 2024, Autonomous Cyber 2023)	漏洞发现与修复闭环	分层式/双环节	RAG 检索外部知识库 (渗透测试书籍)	—	漏洞识别与修复建议双阶段推理 (Counterfactual Prompting)	多工具协同 (Metasploit、Nmap 等)	—	Metasploitable 2 (10 个漏洞)	—
30	CAI[46] (arXiv 2025)	漏洞赏金/渗透任务	多智能体/分层式	—	通过分层智能体协作保留任务阶段状态，支持多场景连续执行	分层智能体协同与自治等级控制	多工具集成	—	CTF / HTB / bug bounty 场景	https://github.com/aliasrobotics/cai
31	Incalmo[47] (arXiv 2025)	企业网络红队任务	多智能体/声明式任务	—	环境状态服务 + 攻击图服务 + C&C 服务器维护多主机资产与状态	高层声明式任务规划 + 领域专用任务智能体执行	多工具协同 (nmap、Metasploit 等)	—	MHBench (40 个多主机环境)	https://github.com/bsinger98/Incalmo
32	EnIGMA[48] (arXiv 2024, ICML 2025)	交互式漏洞利用	智能体工具增强	—	—	交互式推理与工具协作	交互式智能体工具 (gdb、pwntools 等)	—	CTF/漏洞发现场景 (NYU CTF、InterCode-CTF、CyBench、HTB)	https://github.com/SWE-agent/SWE-agent/tree/v0.7 (基于 SWE-agent)
33	Nakano 等人 [49] (arXiv 2025)	全流程	引导推理框架	使用预定义 MITRE ATT&CK 知识	外部代码维护的结构化攻击树 (Structured Attack Tree) 跟踪任务状态 (to-do、in-progress、completed、failed)	攻击树引导任务选择 + 命令生成 (非自主规划)	多工具协同	—	10 个 HTB 机器 (103 个离散子任务)	—
34	Pentest-R1[11] (arXiv 2025)	全流程/推理优化	单智能体/微调模型	采用 500+ 真实多步攻略离线强化学习，强化领域知识内化	—	在线 RL + 离线 RL 两阶段强化学习推理 (GRPO)	—	—	Cybench + AutoPenBench	https://github.com/KHenryAegis/Pentest-R1

续下页注:

表 1 续表

序号	工具（框架）名称	目标阶段	智能体形态	知识增强	记忆管理	规划推理	工具调用	结果验证	评测场景	开源信息
35	Co-RedTeam[50] (arXiv 2026)	真实红队闭环自动化	多智能体	通过长期记忆与执行反馈沉淀攻击经验	将“发现-规划-执行-复现-经验沉淀”全过程写入长期记忆，支持后续任务复用	多智能体协同规划与闭环执行	多工具协同	—	真实红队 workflow	—
36	PentestJudge[51] (arXiv 2025)	评测	法官智能体	—	通过轨迹搜索工具检索调用轨迹与历史操作，为评分保留关键过程信息	树状分层评分规则	工具调用检索/轨迹搜索	✓	智能体轨迹评估任务	—
37	PentestEval[52] (arXiv 2025)	评测	基准/评测框架	—	—	六阶段模块化评测流程	—	×	12 场景, 346 任务	—
38	AutoPenBench[53] (arXiv 2025)	评测	开放基准	—	—	按命令里程碑与阶段里程碑组织评测逻辑	—	×	33 任务的开放 benchmark	https://github.com/lucagioacchini/auto-pen-bench
39	AEAS[54] (arXiv 2025)	测评（漏洞利用可执行性评估）	评测/分析方法	静态分析提取多维特征辅助评估	—	基于静态分析的可利用性评估推理	无需动态执行工具调用	✓	漏洞利用评估任务	作为 PentestAgent 组件集成 https://github.com/nbshenxm/pentest-agent
40	AI-Pentest-Benchmark[55] (arXiv 2025)	评测	开放基准	—	—	面向模型能力差异的系统化评测	—	×	AI-Pentest-Benchmark	https://github.com/isamu-isozaki/AI-Pentest-Benchmark
41	CVE-Bench [56] (arXiv 2025, ICML 2025)	评测	开放基准	Agent 可获得可选漏洞描述 (1-day/0-day 设置)	Agent 通过 prompt 注入漏洞描述作为上下文，探索阶段无长期记忆	ReAct 式动作循环：行动 → 执行 → 观察，部分框架含自我批判与规划	Agent 可调用 sqlmap、ZAP 等工具；基准提供 8 类标准攻击目标	✓	40 个真实 Web CVE (CVSS 9.0)，支持 0-day/1-day 双设置	https://github.com/uiuc-kang-lab/cve-bench
42	HexStrike (https://www.hexstrike.com/)	全流程/产品化	多智能体/产品化系统	集成 150+ MCP tools，具备工具知识增强	—	12 个专用智能体分工协作	150+ MCP 封装工具	—	商业化渗透测试 workflow	https://github.com/0x4m4/hexstrike-ai/
43	PentAGI (https://pentagi.com/)	全流程/产品化	多智能体/产品化系统	基于时序知识图谱构建长期安全知识库	通过长期记忆保留历史命令、输出、决策与策略演化，支持回溯与优化	结合历史经验的策略优化推理	多工具协同	—	可复用的渗透测试 workflow	https://github.com/vxcontrol/
44	Zen-AI-Pentest (https://zen-ai-pentest.dev/)	全流程/产品化	单/混合式产品化系统	—	采用状态管理维持“决策-执行-观察-修正”闭环，支持人在回路干预	ReAct 决策 + 多智能体协作	多工具协同 (nmap、sqlmap、Metasploit、Nuclei 等)	—	人在回路的渗透测试 workflow	https://github.com/SHAdd0WTAka/Zen-Ai-Pentest

注：表中“✓”表示该项在文章中支持或者存在，“○”表示部分支持或者部分存在，“×”表示不支持或者不存在，“—”表示文中未提及该项。

3.2 研究进展概述

基于大模型多智能体的渗透测试研究总体可归纳为三个发展阶段：

技术探索阶段（2023 年）。该阶段主要实现了大模型与渗透测试的初步融合，验证了大模型在渗透测试中的可行性，解决了传统自动化渗透测试工具语义理解不清、逻辑推理薄弱的核心不足，以单一智能体工具与基础框架原型为主。代表性成果包括 PentestGPT[17] 与 AutoAttacker[23]。PentestGPT[17] 代表性地将大模型作为决策单元，整合渗透测试全流程，确立了“大模型 + 渗透测试”的核心技术路线；AutoAttacker[23] 较早探索了多智能体分工协作架构，为后续多智能体框架构建提供了重要借鉴。该阶段实现了大模型与渗透测试的初步融合，打破了传统自动化工具的局限，但尚未完全实现自动化的多智能体协同，测试能力有限，仅能完成单环节、简单场景的渗透测试任务。

协同优化阶段（2024 年）。该阶段聚焦多智能体协同框架构建与协同机制优化，突破了单一智能体的局限，构建了完整的大模型多智能体渗透测试框架，提升了协同测试效率与漏洞发现能力，核心成果以多智能体协同框架与协同机制优化为主。代表性成果包括 PentestAgent[18]、HackSynth[19]、AutoPT[20]、PTGroup[21]、Breachseek[42]、Pentest-ai[43] 等。其中，PentestAgent[18] 系统性地构建了大模型驱动的多智能体协同框架，确立了“决策-执行”双核心架构；HackSynth[19] 构建了首个大规模 CTF 风格标准化评估体系（PicoCTF+OverTheWire 共 200 题），为技术优化提供量化依据；AutoPT[20] 聚焦 Web 场景，实现了 Web 渗透测试的全流程自动化与多步渗透；PTGroup[21] 覆盖 Web 与主机漏洞的综合渗透测试，实现了多类型漏洞环境下的多步渗透；Breachseek[42] 基于 LangGraph 构建面向大规模网络的分布式协同框架；Pentest-ai[43] 融合 MITRE ATT&CK 框架，通过多层平面（工作/数据/控制）设计提升企业网络场景适配能力。该阶段实现了大模型与多智能体的深度融合，技术从单一智能体向多智能体协同演进，构建了较完善的协同机制，能够完成复杂、多步骤的渗透测试任务。

实战适配阶段（2025-2026 年）。该阶段主要面向应用场景适配推动技术落地，提升技术的实战能力。代表性成果包括 VulnBot[22]、Pentest-Chain[30]、PTFusion[28]、xOffense[39]、TermiAgent[38]、vEcho[27]、PenForge[26]、AWE[24] 等。其中，VulnBot[22] 构建任务依赖有向无环图（PTG），实现信息收集-漏洞扫描-漏洞利用三阶段结构化协作；

Pentest-Chain[30] 引入双路 RAG（外部知识库 + 内部经验库）与五智能体链式架构，完善知识支撑与经验复用机制；PTFusion[28] 构建上下文感知知识融合框架，通过动态知识图谱实时建模实体关系，适配动态网络场景；xOffense[39] 在中等规模开源模型（Qwen3-32B）上实现成本与性能的平衡，并结合灰盒提示与反思重规划机制；TermiAgent[38] 提出定位记忆激活与自建武器库机制，并构建了真实世界基准 TermiBench（510 主机，30 个 CVE）；vEcho[27] 提出回响式漏洞主动发现范式，结合认知记忆模块实现未知漏洞挖掘；PenForge[26] 构建动态专家智能体，根据侦察信息即时生成针对性代理，提升复杂场景适配能力；AWE[24] 引入浏览器验证引擎与结构化漏洞分析流程，提升漏洞验证的可信度与可重现性。该阶段实现了应用场景的精细化适配，构建了标准化的性能评估体系，实现了从被动漏洞验证向主动漏洞发现的范式转变，技术逐步走向实战化落地。

3.3 多智能体协作方面研究进展

基于大模型多智能体的渗透测试框架，采用“环境感知-决策规划-执行反馈-迭代优化”为基础的技术路线，通过将大模型作为核心决策单元，结合智能体的自主规划、协作执行与环境交互能力，构建起自动化、智能化的渗透测试流程。结合 2023 年至 2026 年的研究进展，演进路线经历了单一智能体、多智能体协作、动态自适应三个核心发展阶段，实现了从初步自动化到智能化、实用化的迭代升级。

3.3.1 单一智能体架构

单一智能体架构是该领域的早期技术路线，以单个大模型智能体作为框架的唯一决策与执行核心，整合环境感知、漏洞检测、攻击执行、结果反馈等全流程功能模块，依托大模型的逻辑推理与代码生成能力，完成渗透测试从方案规划到结果输出的自动化。该架构的代表性工作为 PentestGPT[17]，其采用单智能体，划分了推理、生成、解析三个核心角色，承担渗透测试全流程相关任务，是单一智能体架构最典型的角色划分方式。该工作系统性验证了大模型智能体在渗透测试领域的可行性。

后续跟进工作围绕 PentestGPT 的局限性展开了针对性优化。RapidPen[33] 聚焦于 IP 到 Shell 的全流程自动化，通过 RAG 检索成功案例知识库与复用历史渗透测试任务树，简化了大模型与工具的交互流程，提升了自动化效率；ARACNE[36] 则提出模块化多 LLM 架构，让规划者、解释器、总结器各司其职，优化了 Shell 环境下的交互逻辑与上下文管理；AutoPT[20] 通过大量实验验证了单一智能体在复杂多步渗透场景下的局限性，重点优化了

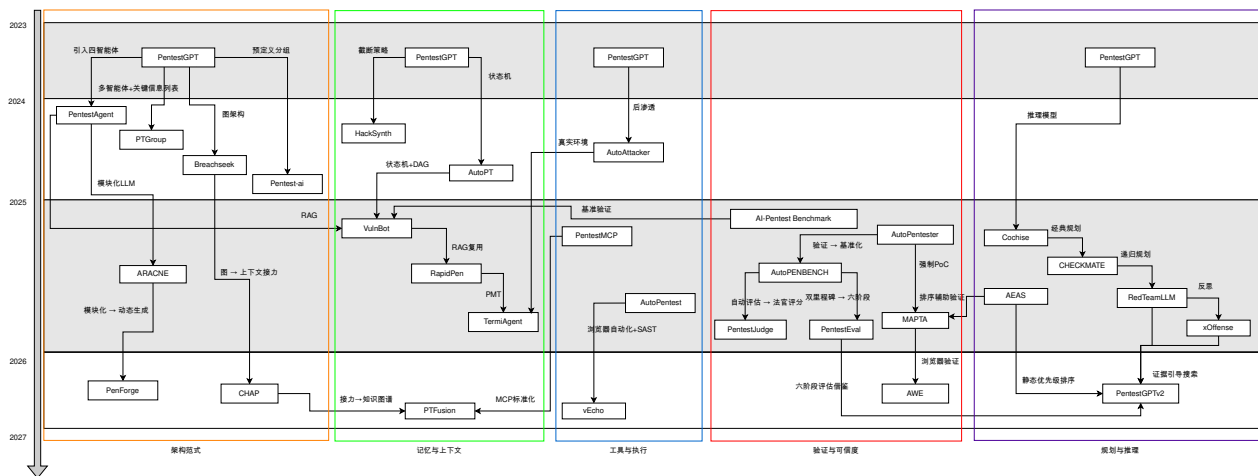


图 2 研究进展演化图

端到端 Web 渗透测试的流程衔接，为后续多智能体架构的研发提供了明确的改进方向。

3.3.2 多智能体协作架构

多智能体协作架构是应对复杂渗透测试场景的核心技术路线，通过一个主智能体将复杂的渗透测试整体任务，拆解为环境扫描、漏洞探测、漏洞利用、结果评估、报告生成等相互独立的子任务，为每个子任务分配专属角色的子智能体，各子智能体依托自身功能模块与知识储备完成专属任务，同时通过信息共享通道实现测试数据、决策结果的实时同步，由主智能体完成协同调度与冲突解决，最终实现整体测试任务的高效、全面完成。

该架构的代表性工作包括 2024 年提出的 PentestAgent[18] 与 2025 年提出的 VulnBot[22]。PentestAgent[18] 设计了侦察、搜索、规划、执行 4 个角色，是多智能体架构早期简洁的典型角色划分模式；VulnBot[22] 则构建了任务依赖有向无环图 (PTG)，将智能体划分为主智能体、探测智能体、利用智能体，体现了多智能体分工的规范化。此后的工作多采用相似的角色划分思路，如 AutoPT[20] 的侦察、漏洞推理、载荷生成、利用尝试四阶段流水线，AutoAttacker[23] 的总结、规划、导航、经验管理分工等。

后续跟进工作围绕协同效率、场景适配性与知识储备展开具体优化。HackSynth[19] 针对协同机制简单、测试结果可靠性不足的问题，构建了 Planner-Summarizer 双模块架构，并建立了首个大规模 CTF 风格标准化评估体系 (PicoCTF+OverTheWire 共 200 题)，为技术优化提供量化依据；PTGroup[21] 采用多提示链技术，为不同角色智能体定制专属提示词，强化了

大模型的决策针对性，解决了多智能体决策偏差的问题；AutoPentester[31] 聚焦企业级场景实用化，优化了子任务拆解机制与智能体角色配置，适配了企业级网络的复杂场景需求；xOffense[39] 通过进攻性知识增强大模型，为各智能体注入 MITRE ATT&CK 框架等安全知识，解决了多智能体安全知识不足的问题；Pentest-Chain[30] 引入双路 RAG 与内部经验管理器，构建渗透测试知识检索库，让各智能体在决策过程中可实时检索漏洞库、攻击案例等知识，进一步提升了决策准确性。此外，Breachseek[42]、Pentest-ai[43] 进一步细化了智能体分工，Pentest-ai 结合 MITRE ATT&CK 框架优化了智能体的攻击策略，提升了多智能体协作的实战能力。PentestMCP[29] 则为多智能体提供了统一的工具调用接口（MCP 协议），降低了智能体与底层工具集成的复杂度。

3.3.3 动态自适应智能体架构

动态自适应智能体架构的核心是通过动态环境感知实时捕捉测试环境变化，自适应调整角色划分或决策策略，实现复杂场景的灵活适配。该路线的代表性工作包括 2025 年的 PTFusion[28] 与 2026 年的 AWE[24]、PenForge[26]。

PTFusion[28] 提出了半去中心化多智能体架构, 设置主智能体负责全局战略规划, 侦察智能体和攻击智能体拥有战术执行自主权, 各智能体可根据上下文自主决定是否继续任务, 平衡集中规划与分散执行之间的矛盾; AWE[24] 采用“协调层 + 专门代理层 + 基础层”的三层架构, 核心引入浏览器验证引擎与结构化漏洞分析流程, 提升了漏洞验证的可信度与可重现性; PenForge[26] 提出专家智能体的动态构建, 在测试过程中根据目标特性即时生

成专用智能体,而非依赖预先定义的角色,实现了从“预定义智能体”向“即时构建”的转变。

后续跟进工作围绕技术落地与实战适配开展了针对性优化:CHAP[25]提出上下文接力机制,通过生成紧凑协议在不同智能体实例间传递状态,优化了长运行测试场景下的上下文管理问题;vEcho[27]针对知识增强框架多聚焦于漏洞验证的局限性,提出回响式漏洞传播机制,从单个漏洞线索出发,通过多轮语义关联和推理,连锁发现更多潜在漏洞,实现了从被动漏洞验证到主动发现漏洞的转变;PentestGPTv2[37]针对优质大模型智能体判定标准模糊的问题,提出任务难度评估(TDA)与证据引导的攻击树搜索(EGATS),为框架的大模型选型与优化提供了理论依据;TermiAgent[38]针对动态自适应框架实战适配性不足的问题,提出定位记忆激活机制与自建武器库(包含1378个Docker化漏洞利用脚本和1077个Metasploit模块,进一步提升了框架的实战能力;RedTeamLLM[41]与xOffense[39]进一步优化了知识增强与动态自适应的融合逻辑,前者强化了攻击场景下的知识适配能力,后者则通过进攻性知识增强与动态任务调度,实现了复杂攻击场景的高效适配。

3.4 领域知识增强方面进展

针对大模型在渗透测试领域的知识缺乏问题,现有研究围绕外部知识引入、领域知识内化、知识高效复用、长时知识留存、深度知识推理等方面开展研究,有效弥补通用大模型的专业知识短板,提升多智能体渗透测试的专业性与准确性。

3.4.1 基于检索增强生成的外部知识实时接入

检索增强生成(RAG)是解决大模型知识陈旧、覆盖不全的基础技术路径,核心是将大模型生成能力与外部知识库检索能力结合,通过外部领域知识的实时接入,实现渗透测试专业知识的“即查即用”[18,21,30]。现有研究构建两类渗透测试知识库:其一为**外部公共知识库**,涵盖CVE漏洞库、MITRE ATT&CK框架、OWASP Top 10、EXP利用脚本库等标准化领域知识[18,20,42];其二为**内部经验知识库**,包含历史渗透测试日志、成功攻击案例、失败经验总结等个性化知识[21,30]。同时采用动态检索机制,根据渗透测试的侦察、漏洞分析、利用等不同阶段,精准检索匹配的领域知识并注入模型上下文,减少噪声干扰,既解决通用大模型的知识幻觉问题,又实现最新漏洞知识的实时获取,可有效应对N-day、0-day漏洞测试场景[18,28]。Pentest-Chain[30]的消融实验表明,引入RAG模块后整体任务成功率提升18.4%,该技术已成为PentestAgent[18]、PTGroup[21]、BreachSeek[42]等框架的核心知识增

强手段。

3.4.2 基于垂类模型微调的域知识深度内化

垂类模型微调实现渗透测试领域知识的深度内化,核心是利用领域专业数据对通用大模型进行针对性训练,将渗透测试的命令语法、漏洞特征、攻击逻辑、场景经验等注入模型权重[37,39]。现有研究的关键在于构建高质量微调数据集,主要包括渗透测试思维链(CoT)数据、漏洞利用案例、攻击路径规划数据等[37,39],并采用LoRA、QLoRA等高效微调方法,在降低计算资源消耗的同时,强化模型的领域专业化推理能力。例如xOffense[39]以Qwen3-32B为基础模型,通过渗透测试CoT数据专项微调,使模型内置攻击知识与漏洞利用逻辑,大幅提升复杂场景下的推理准确性;垂类模型微调技术与RAG形成互补,前者解决了知识的“高效调用”问题,后者则解决了知识的“有无与时效”问题[28,39];xOffense[39]构建攻击知识增强的多智能体框架,通过模型微调注入MITRE ATT&CK等安全知识,强化智能体对复杂攻击场景的适配能力。

3.4.3 基于多智能体的知识分化与共享

多智能体协同架构将复杂渗透测试任务拆解为子任务,通过角色化智能体的分工协作,实现不同类型领域知识的专业化处理与全局共享,为领域知识的高效利用提供了支撑[22,34,42]。现有研究普遍基于渗透测试流程设计角色化多智能体体系,典型角色包括侦察智能体(掌握资产探测、信息收集知识)、规划智能体(掌握攻击路径推理、策略制定知识)、执行智能体(掌握命令语法、EXP利用知识)、验证智能体(掌握漏洞特征、结果校验知识)等[18,22,39],每个智能体专注于特定领域知识的应用,避免单一模型处理全领域知识的能力不足。同时通过消息总线、状态中心、知识图谱等方式,实现智能体间漏洞情报、攻击结果、环境信息等知识的实时共享,形成全局知识视图[22,29]。典型代表为VulnBot[22],通过多类智能体的协同与知识共享,实现攻击知识的闭环迭代。

3.4.4 基于知识图谱的领域知识深度推理与动态适配

知识图谱可将零散的渗透测试知识组织为结构化图谱,结合自适应算法可实现知识的深度推理与动态场景适配[28,29]。现有研究通过构建知识图谱,挖掘各要素间的关联关系,实现漏洞关联分析与攻击路径推理。例如,PTFusion[28]针对传统框架“知识碎片化、无法实现多源知识融合”的问题,将知识图谱与上下文感知融合,在测试过程中基于动态知识图谱实时构建实体关系,强化Web渗透测试

中的知识深度推理能力。此外, 结合自适应规划算法, 可基于知识图谱的推理结果与实时环境状态动态调整攻击策略。例如 AWE[24] 根据获取的凭证、服务状态等知识, 自动将测试模式从匿名测试升级为认证测试, 实现领域知识与动态 Web 场景的精准适配; vEcho[27] 提出结构化漏洞分析流程, 将领域知识嵌入分析流程而非仅依赖检索, 通过知识图谱与环境感知的融合, 实现未知漏洞的主动推理与发现。

3.5 短期记忆管理方面进展

大模型原生记忆存在上下文窗口有限、长流程信息易丢失、多步骤任务状态衔接断裂等问题, 难以适配渗透测试多阶段、长链路、需连续状态感知的任务特性。现有研究围绕记忆结构化、上下文传递、记忆动态更新、记忆精准调用等技术路线开展研究, 针对性弥补短期记忆短板, 保障渗透测试全流程的信息连续性与状态一致性。

3.5.1 结构化记忆建模

针对大模型原生记忆的无序性问题, 研究者们通过构建标准化结构化记忆模块, 将渗透测试过程中的瞬时信息按类型、阶段分类存储, 让智能体可快速提取关键记忆信息, 避免无关信息干扰短期记忆。PentestGPT[17] 提出了渗透测试任务树 (PTT) 的概念, 用树形结构维护测试上下文, 为解决对话丢失问题提供了基础框架。后续研究将记忆内容划分为资产信息、服务状态、漏洞详情、Payload 尝试、执行结果、权限等级等核心维度 [22, 24, 25], 采用统一状态模型、键值对存储、结构化日志等方式组织记忆数据。例如 AWE[24] 搭建统一状态模型, 将动态 Web 测试中的输入点、认证状态、Payload 执行记录等短期记忆结构化存储, 智能体可按测试阶段精准调取对应记忆; VulnBot[22] 则将攻击失败原因、成功步骤等短期记忆与漏洞类型关联存储, 为后续同类型漏洞测试提供可复用的瞬时经验, 有效解决了原生短期记忆信息杂乱、调取效率低的问题。

3.5.2 上下文中继传递

针对大模型上下文窗口有限、长链路渗透测试中前期记忆易丢失的问题, 研究者提出了上下文中继机制 [25], 实现跨阶段、跨步骤的上下文信息高效传递, 保障长时程测试任务中记忆的连续性。该技术核心是对全流程上下文进行轻量化提取与精准中继, 剔除冗余信息, 保留核心状态与关键操作记录, 并在每个测试阶段开始前, 将相关历史上下文注入模型当前推理窗口。典型代表为 CHAP[25], 专为长时程渗透测试设计了上下文中继机制, 通过提

取攻击路径、已获权限、未利用漏洞等核心短期记忆, 实现数小时连续测试的上下文无缝传递, 避免因窗口限制导致的前期记忆丢失; PentestAgent[18] 则在规划智能体与执行智能体的协作中, 通过中继精简后的上下文信息, 确保执行环节精准承接规划阶段的短期记忆, 实现任务步骤的连贯推进。

3.5.3 动态记忆更新与修剪

渗透测试是动态迭代的过程, 短期记忆需随测试推进实时更新, 同时需要剔除失效、冗余信息, 避免无效记忆占用有限的短期记忆空间。现有研究主要通过“动态更新 + 主动修剪”实现短期记忆的高效管理。在动态更新方面, 智能体每完成一个测试步骤, 便将执行结果、新发现的漏洞或资产、权限变化等关键信息实时写入短期记忆 [22, 38]。例如 TermiAgent[38] 提出的定位记忆激活机制, 会将端口扫描结果、漏洞验证状态等瞬时信息实时更新至短期记忆, 为后续攻击路径选择提供最新依据; 在主动修剪方面, 研究通过设置记忆有效期、按重要性分级筛选等方法, 自动剔除已利用的 Payload、已排除的无漏洞资产等失效短期记忆 [24, 25], 确保短期记忆中始终保留高价值、高相关性的信息, 提升模型推理效率。例如 CHAP[25] 在上下文接力过程中通过生成紧凑协议 (compact protocol), 将攻击路径、已获权限、未利用漏洞等核心上下文进行压缩编码, 原始上下文中已被利用的 Payload、已排除的无漏洞资产等失效信息被主动剔除, 仅保留高价值信息传递至新的智能体实例。AWE [24] 则采用统一状态模型管理输入点、认证状态和载荷尝试记录, 在上下文接力时通过“短期记忆 + 长期知识库”的分层设计, 由验证引擎对已尝试过的无效载荷进行标记和过滤, 确保传递的短期记忆始终保持高相关性 with 高价值。这些研究共同表明, 动态更新与主动修剪相辅相成, 前者通过实时写入确保短期记忆的时效性与完整性, 后者通过有效期管理、重要性分级、上下文压缩等方式剔除冗余, 二者协同支撑渗透测试智能体在复杂、长周期任务中维持高效决策能力。

3.5.4 记忆激活与精准调用

为解决大模型在推理中难以精准调取相关短期记忆的问题, 研究者引入记忆激活触发机制, 根据当前测试任务与场景特征, 精准匹配并激活相关短期记忆, 让模型在特定环节快速调用对应信息。该技术为短期记忆添加场景、任务、漏洞类型等标签 [29, 38], 当智能体进入某一测试阶段或面对特定漏洞类型时, 通过标签匹配自动激活相关记忆。例如 TermiAgent[38] 的记忆激活机制, 在探测到 MySQL 服务时, 会自动激活短期记忆中相关的 MySQL 漏

洞扫描记录、Payload 尝试结果等信息,为后续漏洞利用提供参考;PTFusion[28]则结合动态知识图谱的关联推理,在规划攻击路径时,激活短期记忆中与目标资产、漏洞类型匹配的执行记录,实现短期记忆与当前任务的精准适配,避免记忆调用的盲目性。

3.6 推理能力提升方面进展

传统自动化渗透测试工具(如 Metasploit、Nessus)依赖预设的漏洞库与攻击脚本,缺乏对复杂场景的动态分析和推理能力,难以应对 0-day 漏洞、多层级网络拓扑、动态防护策略等真实场景的挑战。多智能体系统通过将复杂渗透任务拆解为专业化子任务,实现分工协作,进一步提升了系统的推理效率与覆盖范围。基于大模型多智能体渗透测试的推理方式从早期单一大模型的链式推理,逐步演进为多智能体协同推理、结构化规划推理与自适应动态推理。

3.6.1 多智能体协同推理

针对单一大模型的推理瓶颈,核心优化手段是按渗透职能拆分专业化智能体,将高维复杂的渗透决策拆解为低维子任务,通过分工降低单一智能体决策复杂度,通过协作提升推理的覆盖度与连贯性,成为 2024 年该领域的研究核心。例如,Breachseek[42]的多智能体系统拆分出侦察、漏洞分析、漏洞利用等专用智能体,通过子任务拆解与结果聚合缓解大模型上下文窗口限制,提升长时程渗透的推理连贯性;Pentest-ai[43]基于 MITRE ATT&CK 框架构建多智能体系统,将攻击战术与智能体能力精准对齐,实现标准化、可解释的协同攻击推理,解决多智能体决策冲突问题。2025 年,协同推理机制进一步向状态共享与动态调度升级。VulnBot[22]设计全流程专业化智能体并模拟红队分工模式,通过任务流调度实现多智能体高效协同,提升多主机、多漏洞场景的推理覆盖度;MAPTA[34]面向 Web 渗透构建分布式多智能体系统,强化智能体间的漏洞关联推理与联合探测能力,解决多层 Web 架构的推理盲区;Cochise[35]将该机制应用于企业内网渗透,通过智能体间攻击状态共享,实现 Active Directory 网络的自动化攻击路径规划,验证了协同推理在复杂企业环境中的有效性。

3.6.2 结构化推理

为解决大模型推理的随机性、不可解释性问题,研究者将多提示链并行、形式化建模、攻击路径规划等手段与大模型多智能体结合,提升渗透测试推理的可控性、可复现性与安全性。例如,PTGroup[21]

采用多提示链并行推理模式,从不同维度引导智能体生成攻击策略,通过投票与融合机制降低单一大模型的推理偏差,提升决策鲁棒性;AutoPT[20]通过对端到端 Web 渗透推理的系统性评测,明确了模块化、结构化、可观测的优化方向,为结构化推理研究提供理论指导;CHECKMATE[40]与经典 AI 规划相结合,提出形式化攻击路径生成机制,通过明确前提条件、动作规则与目标约束,实现严谨、可验证的攻击规划;Cochise[35]则在企业内网渗透场景中,通过形式化建模实现攻击路径的可审计与可校验,进一步验证了结构化推理在提升推理可靠性中的作用。

3.6.3 自适应动态推理

针对真实网络环境动态多变、防护异构、漏洞复杂的特点,研究者提出了环境感知型自适应智能体及动态策略调整机制,让多智能体系统根据目标环境反馈实时调整推理逻辑与攻击策略,实现专家智能体的按需构造与推理错误的有效恢复,大幅提升系统对复杂真实环境的泛化能力。例如,AWE[24]提出了自适应智能体机制,能够基于目标环境的探测反馈动态调整渗透策略与推理逻辑,实现对异构 Web 系统的精准推理;PenForge[26]实现专家智能体的按需动态构造,可针对特定目标、漏洞类型即时生成专用推理与执行能力,有效适配 0-day 漏洞与新型漏洞场景。在推理能力拓展与优化总结方面,vEcho[27]通过深度交互推理将渗透推理从“漏洞验证”推向“主动发现”,拓展了推理的覆盖范围与应用边界;RapidPen[33]基于任务规划通过动态推理策略调整,实现从 IP 到 Shell 的全流程自动化渗透,验证了自适应推理在快速渗透场景中的有效性;RedTeamLLM[41]引入动态计划修正机制,子任务失败时不终止整体,而是自动调整后继续攻击树,寻找替代路径继续推进,强化了智能体的环境感知与动态决策能力。

3.7 工具调用优化方面进展

渗透测试涉及多种专用工具,涵盖信息收集(如 Nmap、Whois)、漏洞扫描(如 Nessus、OpenVAS)、漏洞利用(如 Metasploit)、权限维持(如 Empire)等全流程,不同工具的适用场景、参数配置、使用逻辑存在显著差异。大模型强大的自然语言理解与逻辑推理能力,能够实现工具的自动选择、参数配置与结果解析;多智能体系统则通过分工协作,实现多工具的协同调用与任务闭环。然而,大模型多智能体在渗透测试工具调用中仍面临三大核心问题:一是工具调用的合理性不足,大模型易因缺乏领域知识导致工具选择错误、参数配置不当;二是多智能体与多工具的协同联动机制不完善,易出现工具

调用冗余、任务冲突；三是工具调用的自适应能力欠缺，难以根据目标环境反馈、工具执行结果动态调整调用策略。相关研究围绕上述问题展开针对性优化，逐步推动工具调用向智能化、协同化、自适应方向发展 [17, 20, 23]。

3.7.1 大模型驱动的自动化工具调用

2023 年，研究者在基于大模型的工具调用上取得了突破，实现了工具选择、参数配置、结果解析的自动化，验证了大模型在工具调用中的可行性与有效性，为渗透测试工具的自动化调用奠定了基础。代表性工作为 PentestGPT[17]，较早构建了大模型驱动的自动化渗透测试工具调用框架，实现了渗透测试全流程的工具自主调用。该框架通过提示工程引导大模型，根据渗透任务阶段（信息收集、漏洞扫描、漏洞利用）动态选择适配工具，自动生成工具调用指令与参数配置，同时解析工具执行结果，为后续任务阶段的工具调用提供决策依据，摆脱了传统渗透测试中工具调用对人工的依赖。AutoAttacker[23]进一步强化了大模型的工具调用能力，聚焦攻击工具与漏洞利用的适配性，通过优化提示策略，提升了大模型对 Metasploit 等漏洞利用工具的调用准确性，实现了攻击载荷与工具参数的精准匹配，验证了单大模型在工具调用中的实用价值。

这一阶段的研究初步实现了工具调用的自动化，但仍存在明显局限：单一大模型的上下文窗口限制导致长流程工具调用的连贯性不足，难以应对多工具协同调用场景，且工具调用的容错性较差，易因工具执行失败导致任务中断 [20, 23]。

3.7.2 多智能体协同的多工具联动调用

针对单一大模型工具调用存在的局限性，2024 年以来，研究重点转向多智能体协同驱动的工具调用，通过按渗透职能拆分智能体，实现工具调用的分工协作与多工具联动，提升复杂场景下工具调用的效率与覆盖度。例如，Breachseek[42] 将渗透任务拆解为侦察、漏洞分析、漏洞利用等专用智能体，每个智能体负责特定类型工具的调用与管理：侦察智能体调用 Nmap、Whois 等信息收集工具，漏洞分析智能体调用 Nessus 等扫描工具，漏洞利用智能体调用 Metasploit 等攻击工具，通过智能体间的结果共享与任务调度，实现多工具的协同联动，缓解了单一大模型的上下文限制，提升了工具调用的连贯性。Pentest-ai[43] 基于 MITRE ATT&CK 框架构建多智能体系统，将攻击战术与工具调用能力对齐，每个智能体对应特定 ATT&CK 战术的工具调用，实现了工具调用的标准化与规范化，避免了工具调用冗余与任务冲突，提升了多工具协同调用的效率。

2025 年以来，多智能体工具调用的协同机制进一步优化，更注重智能体与工具的深度融合。例如，VulnBot[22] 设计了侦察、扫描、利用、权限维持等全流程智能体，每个智能体集成对应类别的渗透工具，通过任务流调度实现工具调用的无缝衔接，通过模拟红队人员的工具使用逻辑，提升了多工具协同调用的合理性。MAPTA[34] 面向 Web 渗透场景，构建分布式多智能体系统，强化智能体间的工具调用协同，实现信息收集工具与 Web 漏洞扫描工具、漏洞利用工具的联动，解决了多层 Web 架构下工具调用的适配性问题。AutoPentester[31] 则通过多智能体分工实现工具调用的并行处理，提升了大规模网络渗透中工具调用的效率。

3.7.3 精准可调控的工具调用

随着研究的深入，工具调用的优化重点转向提升调用合理性、容错性与可验证性，通过结构化规划、多提示优化、结果反馈调整等技术，解决大模型工具调用中的幻觉、参数错误、执行失败等问题，实现工具调用的精准化与可调控。

PTGroup[21] 采用多提示链并行优化工具调用推理，通过不同提示链引导智能体生成工具调用策略，结合投票融合机制选择最优调用方案，降低大模型工具选择与参数配置的偏差，提升了工具调用的合理性。AutoPT[20] 提出通过工具调用日志记录与结果反馈，实现工具调用的动态调整，提升了工具调用的容错性。CHECKMATE[40] 结合经典 AI 规划，形式化规划工具调用，通过明确工具调用的前提条件、动作规则与目标约束，实现工具调用路径的精准规划，避免无效工具调用，提升了工具调用的效率与可验证性。2025 年以来，工具调用的针对性优化进一步深化。RapidPen[33] 支持动态调整工具调用顺序与参数，如工具 nmap 超时后替换为 rustscan，实现工具调用的高效衔接，大幅提升了渗透测试的速度。ARACNE[36] 专注于 Shell 层面的工具调用，优化大模型对命令行工具的调用能力，实现 Shell 工具的自动化调用与结果解析，适配 Web 渗透与内网渗透场景。Cochise[35] 在企业内网渗透场景中，优化智能体对 Active Directory 网络中相关工具的调用策略，通过工具调用结果的实时分析，动态调整调用方案，提升了复杂内网环境下工具调用的适配性。

3.7.4 动态环境下的自适应工具调用

针对真实网络环境动态多变、防护策略异构的特点，2026 年以来，研究重点转向自适应工具调用，根据目标环境反馈、工具执行结果、漏洞类型变化，实现工具调用策略的动态调整，自主选择适配工具与参数，提升工具调用的泛化能力与鲁棒性。

AWE[24] 设计了自适应智能体核心模块,能够实时感知目标环境的防护策略与系统特性,动态调整工具调用策略:针对防护严格的目标,优先调用隐蔽性强的扫描工具与攻击工具;针对异构 Web 系统,自动选择适配的漏洞扫描工具与利用工具,实现工具调用的环境自适应。PenForge[26] 实现了专家智能体的按需动态构造,每个专家智能体对应特定类型工具的调用能力,可根据目标漏洞类型、系统环境,即时生成专用工具调用智能体,实现工具调用的精准适配,尤其适用于 0-day 漏洞与新型漏洞场景。此外, vEcho[27] 通过深度交互推理与认知记忆,实现了从被动验证到主动发现的转变,其基于前期工具调用结果动态生成扫描策略,提升了未知漏洞挖掘中工具调用的针对性与有效性。RedTeamLLM[41] 强化智能体的工具调用自适应能力,通过实时分析工具执行反馈,动态调整工具调用顺序与参数,提升了真实红队演练中工具调用的鲁棒性。PentestGPTv2[37] 系统总结了真实场景下智能体工具调用的关键优化点,明确了工具集成的完整性、参数配置的准确性、执行结果的解析能力是提升工具调用效果的核心,提出了针对性的优化策略,为自适应工具调用的实用化提供了指导。

3.8 测试评估方面进展

随着大语言模型驱动的多智能体渗透测试技术从单步指令执行向自主化、协作化、长时推理方向发展,其测试与评估体系也逐步突破传统以“是否获取目标权限”为核心的粗粒度评价模式,向标准化基准、多维度扩展的精细化评测演进。

3.8.1 标准化评估

早期评估工作多聚焦端到端渗透任务成功率,难以定位信息收集、漏洞理解、工具调用、权限提升等环节的能力瓶颈,且评测环境零散、实验配置差异较大,导致不同多智能体框架之间缺乏公平可比的评价基础。为解决评测碎片化问题, PentestEval 基准 [52] 将渗透测试流程划分为信息搜集、漏洞探测、漏洞利用、权限维持等标准化阶段,采用模块化与阶段级联合度量方式,实现对多智能体决策连贯性、工具执行准确率、攻击路径合理性的细粒度定位与量化对比,为多智能体协作架构的测评奠定了基础。AutoPenBench[53] 则进一步面向智能体设计标准化漏洞测试靶场,通过可控可复现的实验环境消除场景差异带来的评价偏差,支持对大模型理解漏洞原理、动态构造攻击载荷、自适应调整攻击策略等关键能力的系统性评测。与此同时, AI-Pentest-Benchmark[55] 通过系统分析不同大模型在渗透测试各环节的能力差异,针对性提出优化策略,进一步丰富了标准化评测的技术路径,为多智能体架构

选择与模型优化提供了更具针对性的参考。

3.8.2 多维度扩展

在评测维度扩展方面, PentestJudge[51] 突破传统仅关注任务达成率的局限,引入面向作战需求的智能体行为判定机制,从操作合规性、步骤合理性、风险可控性等角度对多智能体行为进行约束性评估,有效提升了自动化渗透测试在真实红队任务中的可信性与可部署性。AEAS 可执行漏洞评估系统 [54] 聚焦渗透测试中漏洞利用的有效性与可操作性评估,填补了传统评测中“漏洞探测与实际可利用性脱节”的技术空白,通过量化漏洞利用的成功率、稳定性及影响范围,为多智能体渗透测试的评估提供了更贴合实际应用场景的补充维度。针对复杂企业内网与长时持续渗透场景, Happe 等利用 Cochise[35] 框架在 AD 域环境下开展自主渗透测试评估,验证了多智能体在内网横向移动、多级权限链规划中的实际能力边界,揭示了现有模型在复杂拓扑推理与长期策略规划上的不足; CVE-Bench[56] 则针对智能体在真实 Web 应用漏洞环境里的漏洞利用能力开展测评; CHAP[25] 与 TermiBench 基准 [38] 则进一步面向长时运行渗透任务,构建了面向上下文持久化、会话状态维护、记忆激活与多轮交互鲁棒性的专项评测指标,精准刻画了多智能体在超长交互链中出现的上下文遗忘、决策漂移与协作失效问题。

此外,部分研究将评测模块与多智能体推理深度耦合,如 HackSynth 内置一体化评估闭环 [19], RefPentester[44] 引入自反思式评测机制,使评估从被动验证转向驱动智能体策略进化的关键环节。总体来看,当前大模型多智能体渗透测试的评估研究已形成较完善技术体系,但面向动态对抗环境、多智能体协同、跨场景泛化等方面的能力仍处于初步探索阶段。

3.8.3 评测覆盖度分析

为了进一步对比不同测评基准的复用程度和覆盖情况,表 2 汇总了典型评测工作的核心评测对象、现有执行框架覆盖情况以及覆盖度判断。

从覆盖度来看, AutoPenBench[53] 因其采用轻量化容器化漏洞任务、Flag 里程碑式的自动化指标,以及易于集成的半结构化实验环境,成为当前执行类框架中覆盖最充分的评测基准, VulnBot[22]、CHAP[25]、xOffense[39]、Pentest-R1[11] 等均直接采用或对照验证,覆盖度评价为“高”; TermiBench[38] 以真实多主机、多服务器环境及 Shell/root 为成功准则,虽然已有本身的 TermiAgent 直接评测,且 VulnBot[22]、PentestGPT[17] 作为对照,但目前仍集中在少

表 2 评测/基准类工作与执行类框架的覆盖度对照表

序号	评测/基准类工作	核心评测对象	现有执行类框架中的覆盖情况	覆盖度	判断
1	TermiBench[38] (arXiv 2025)	真实多主机、多服务器、以 Shell/root 为成功准则的实战渗透对象	TermiAgent 直接评测；报告中还采用 VulnBot、PentestGPT 作为对照	中	已有代表性对照，但目前仍集中在少数头部框架，广度不够
2	AutoPenBench[53] (arXiv 2025)	容器化漏洞任务、Flag/里程碑驱动、简单服务/半结构化对象	VulnBot、CHAP、xOffense、Pentest-R1，报告里还出现 TermiAgent / PentestGPT 对照	高	这是当前执行类框架覆盖最充分的对象之一
3	AI-Pentest-Benchmark[55] (arXiv 2025)	VulnHub/VM 风格单机场景（偏 Flag/任务完成）及模型能力差异下的系统化渗透任务	VulnBot、xOffense	中偏低	有复用，但参与框架仍少，覆盖度中等偏低
4	PentestJudge[51] (arXiv 2025)	智能体轨迹本身：Operational Objectives / Opsec / Tradecraft	理论上可覆盖任意执行框架，但现有材料里主要是通用轨迹评估，并未形成对大量具体框架的系统覆盖	低	评的是“过程质量”而不是环境对象，和执行框架评测还没真正实现
5	PentestEval[52] (arXiv 2025)	六阶段模块化能力对象、12 场景 346 任务	现有表格里尚未看到明确有哪些执行类框架系统跑过它	低	更像“提出评测协议”，不是已被广泛复用的共同基准
6	AEAS[54] (arXiv 2025)	exploit 可执行性/可操作性对象，而非端到端渗透环境	主要作为 PentestAgent 组件集成	低且窄	覆盖的是“PoC/exploit 筛选”这一窄对象，不覆盖完整攻击链
7	CVE-Bench[56] (arXiv 2025)	真实 Web CVE 漏洞发现/验证对象	PenForge 明确使用	低	真实度高，但在当前执行类框架集合里复用很少

数头部框架，覆盖度评为“中”；AI-Pentest-Benchmark[55] 聚焦 VulnHub/VM 风格单机场景及模型能力差异下的渗透任务，虽被 VulnBot[22]、xOffense[39] 采用，但参与框架数量有限，覆盖度中等偏低。

相比之下，PentestJudge[51] 以智能体轨迹过程质量为评测对象，理论上可适配任意执行框架，然而现有工作仍以通用轨迹评估为主，尚未形成对大量具体框架的系统化覆盖，覆盖度较低。PentestEval[52] 提出了六阶段模块化能力评测协议与 12 场景 346 项任务，但尚未见到执行类框架系统运行该基准，覆盖度同样偏低。AEAS[54] 专注于漏洞利用的可执行性与可操作性评估，仅作为 PentestAgent[18] 的内部组件集成，不覆盖完整攻击链，覆盖范围“低且窄”。CVE-Bench 面向真实 Web CVE 漏洞发现与验证，虽有 PenForge[26] 明确使用，但在当前主流执行框架集合中复用极少，覆盖度低。

综合来看，当前评测基准体系呈现“两端分化”的特征：面向 CTF/容器化 Flag 驱动的基准已获得广泛采纳，而面向真实企业内网、长时 Active Directory 域、过程质量、漏洞利用可操作性、真实漏洞环境等“实战属性”更强的基准，与执行类框架之间的整合仍处于初级阶段。这一现状表明，未来评测体系的发展需要面向高真实度、高复杂度场景，形成覆盖度更均衡、结果更具横向可比性的

评测生态。

4 智能体时代软件渗透测试的挑战

尽管技术进展显著，基于多智能体的渗透测试在实际部署与应用中仍面临多重瓶颈与挑战。

4.1 长流程任务支撑不足

渗透测试是由多个相互关联的子任务串联构成的复杂过程，通常具有链路长、状态多、阶段依赖强等特点，不同子任务之间往往存在明显的上下文依赖关系，前序任务获得的信息会直接影响后续任务的判断与执行。智能化渗透测试需要在多轮交互与多工具调用过程中持续维护资产信息、服务状态、凭据线索、已尝试载荷及阶段目标之间的关联，一旦这些信息不能被稳定保留，系统便容易出现重复侦察、错误转移或攻击链中断。例如，在 HackSynth 的测试案例中，模型需在 10 个步骤中维护子域名 devops.example.com 与 Jenkins 未授权访问漏洞的关联，但 GPT-4 在第五步后即丢失该上下文，转而盲目扫描其他端口 [19]。PentestGPT 虽通过模块化设计缓解此问题（各模块缓存局部状态），但在多目标攻击链中仍会出现上下文断裂 [17]。因此，现有工作普遍引入任务树、状态机、外部状态存储、摘要中继和记忆激活等机制，达到“跨阶段状态保持”。然而，更深层的问题在于大模型本身的检索和推理能力，随着上下文长度不断增加，大模型存在“lost-in-the-middle”现象——当关键信息被埋藏

在长上下文的中间位置或分散在多个干扰项中时,模型的检索和推理性能会显著下降,这种机制性缺陷导致即使在理论上可容纳 128K 乃至 1M token 的上下文窗口中,大模型仍难以有效利用早期发现的关键线索。因此,如何使大模型能够对长流程上下文中的关键信息进行有效提取、关联与串联,从而支撑多阶段任务的持续推理,是提升渗透测试整体成功率的关键问题。

4.2 复杂任务处理能力不足

现有研究虽通过模块化流程、多智能体分工、结构化规划和反思机制提升了复杂任务处理能力,但相关评测与系统分析仍表明,跨阶段信息传递损失、工具输出噪声以及目标状态漂移会持续放大推理误差,导致端到端成功率和稳定性受限 [20, 36, 37, 40]。典型表现有两个:一是**多跳推理失败**。例如,提权需综合分析内核版本(通过 `uname -r`)、SUID 文件(通过 `find / -perm -4000`)和服务配置(如 MySQL 以 root 运行),但大模型通常孤立处理这些线索,导致仅 12% 的测试案例能完成全链条利用 [19]。二是**“幻觉”引发严重错误**。例如,测试中大模型会生成虚构的 Metasploit 模块,因模块路径错误导致攻击失败。更严重的是,模型可能误判漏洞可利用性,例如在 CVE-2022-22963 (Spring Cloud Gateway 漏洞)的测试中,大模型在未验证 `spring.cloud.gateway.enabled` 配置的情况下直接发起 RCE 攻击,触发误报。从本质上看,这种幻觉源于大模型的推理机制——模型擅长根据已有模式生成看似最可行的解释,但无法直接观察系统真实状态,导致置信度与实际可利用性之间存在根本性脱节。在复杂业务逻辑场景中,AI 模型仍易将合法行为误判为攻击,平均误报率维持在 15%-25% 之间。当前智能化渗透测试对复杂任务处理能力不足并不只是基础模型能力问题,更与状态建模、规划表示、结果验证及纠错闭环等多个流程密切相关。

4.3 渗透测试经验难以复用

渗透测试高度依赖操作经验、工具使用习惯与失败后的修正策略,但现有大模型系统对这类经验的吸收和复用仍然较为有限。已有研究虽然开始尝试通过内部经验管理器、历史任务树复用、成功案例检索、动态记忆和漏洞利用武器库等方式,将过往测试过程中的命令、载荷、攻击路径和修正记录保留下来,以辅助后续任务决策 [22, 23, 30, 33, 38],但这些经验大多仍以日志、案例库、脚本库或检索片段的形式存在,尚未稳定沉淀为可组合、可迁移、可持续优化的能力模块。其结果是,系统在面对新目标时往往仍主要依赖基础模型的预训练知识和当前上

下文中的临时注入信息,难以像熟练测试人员那样调用经过长期积累的策略模板、工具协同范式和错误恢复经验。尤其在多阶段攻击链和异构环境中,经验无法有效模块化会进一步削弱系统的迁移能力与自我增强能力,限制智能化渗透测试从“单次任务求解”走向“持续能力积累”的发展空间 [24, 26, 37, 39]。

4.4 经济成本成为规模化障碍

基于 GPT-4 的测试单次成本高达 131.5 美元(以平均 5 次 API 调用/任务计算),而本地部署的 70B 模型需 4×A100 GPU (每小时成本约 3 美元),且响应延迟(平均 12 秒/命令)难以满足实战需求。更严峻的是,模型微调的数据稀缺。渗透测试的对抗性本质导致高质量数据集(如真实企业网络的漏洞利用链)难以合法获取,现有研究多依赖公开 CTF 赛题(如 HackTheBox),与实际环境的复杂性差异显著。

4.5 评估测试基准局限性日益凸显

现有评测体系正在从零散案例走向标准化 benchmark,但其局限性也日益显现。一方面,许多基准仍以 CTF 或受控靶场为主,任务目标、环境噪声和防御约束与真实渗透工作并不完全一致,例如,PT-Bench 虽覆盖 OWASP 十大漏洞,但缺乏对云原生场景(如 Kubernetes RBAC 配置错误)和 IoT 协议(如 CoAP 的 UDP 漏洞)的覆盖;另一方面,评估指标仅关注“是否获得 Shell”或最终任务完成率,而忽视侦察覆盖、阶段衔接、错误恢复和子任务完成质量等关键能力(如 Nmap 扫描的端口覆盖率、目录爆破的路径深度等),从而高估系统的端到端表现。例如,某次测试中大模型通过 `admin.php` 弱密码获取 Webshell,但未发现同一服务器上的 `/backup` 目录泄露数据库凭证,导致评估得分虚高。这种评估标准的粗粒度使得工具性能难以横向对比,也掩盖了实际能力的真实短板。近期研究虽已开始引入更贴近实战的基准与分阶段评测设计,例如 TermiBench[38]、AutoPenBench[53] 以及阶段级评测框架,但 benchmark 真实性、跨环境可比性和指标粒度仍是制约结论外推的关键因素。因此,对智能化渗透测试系统的评估需要同时关注场景真实性、过程性指标,而不宜仅凭单一终局指标判断系统优劣。

5 未来发展展望

5.1 工具使用知识的深度整合

当前基于大模型的多智能体渗透测试框架大都具备规划和决策能力,即知道该做什么,但在具体执行层面(如工具用法、参数组合、多工具串联)还

存在不足。PenForge 论文中指出工具使用知识不足会影响漏洞利用效果 [26]; AWE 论文也指出需要基于浏览器进行漏洞验证, 即通过真实载荷执行, 观测 JavaScript 执行、DOM 变化等确定性信号, 才能确认漏洞的可利用性, 因此提出了结构化分析流程的解决思路 [24]。近期 Claude Code、OpenClaw 等智能体框架发展迅速, 其中采用的 skill 机制也为提升工具的使用能力提供了思路, 未来可以将 skill 文件作为人的经验载体, 提升智能体使用工具的能力, 弥补当前自动化工具调用的不足。

5.2 0-day 漏洞与未知场景的适应能力

现有工具擅长已知漏洞的分析和利用, 但对 0-day 漏洞成功率较低, 例如, PenForge 在 0-day 设置下成功率只有 20% [26]。针对此问题, vEcho 基于回响式漏洞传播尝试从被动验证转向主动发现 [27], PenForge 在测试过程中根据目标特性动态构建专用代理 [26], 这些方法一定程度上提升了渗透测试对动态复杂场景的适应能力。未来基于大模型多智能体的渗透测试框架可以与大模型驱动的漏洞挖掘框架、漏洞利用框架深度整合, 打通漏洞扫描 [57]、漏洞挖掘 [58]、漏洞利用 [59]、利用评估的全链路协同。

5.3 持续学习与经验沉淀机制

渗透测试系统的发展, 不仅需要获取外部知识, 更需要具备从任务执行过程中持续沉淀经验并将其转化为稳定能力的机制。现有研究已经在这一方向进行了初步探索, 例如 AutoAttacker [23] 引入经验管理器缓存成功动作以支持后续复用, Pentest-Chain [30] 设计内部经验管理器实现跨任务经验联想, RapidPen [33] 通过复用历史成功渗透任务树和成功案例数据提升新任务效率, AWE [24] 与 vEcho [27] 分别构建短期/长期记忆系统以及临时/永久知识库, 用于保留漏洞验证过程中的关键中间结果与策略经验, TermiAgent [38] 则通过自建漏洞利用武器库强化真实环境中的经验复用能力。未来可进一步围绕经验表示标准化、经验质量评估、跨任务迁移复用以及经验驱动的安全增量更新机制展开研究, 在保证可审计性与可控性的前提下, 推动渗透测试智能体从“单次任务执行”走向“持续能力演化” [37, 39, 41]。

5.4 仿真环境的真实性与标准化

虽然目前有很多研究开展多智能体渗透测试框架测评的工作, 如 PentestJudge [51]、TermiBench [38]、PentestEval [52]、CVE-Bench [56]、AutoPenBench [53] 等, 但当前针对渗透测试智能体的评估标准, 大多将“漏洞利用是否成功”作为

核心指标, 而对于未知网络拓扑的探索能力、目标环境的信息收集能力、攻击路径规划能力以及多阶段决策能力等关键能力尚未得到充分覆盖。此外, 当前许多测试集基于已知漏洞和现成靶场构建, 这类题目通常已经在网络上存在公开解法, 其相关内容也可能出现在大模型的训练语料中, 从而影响评估结果的真实性与区分度。因此, 如何避免“记忆命中”带来的干扰, 构建更贴近真实场景、更具挑战性和泛化性的评估体系, 是未来大模型多智能体渗透测试能力评估的重要研究方向。

5.5 多种技术路径的融合发展

与基于强化学习的渗透测试路线融合。传统强化学习路线因状态空间爆炸与奖励稀疏而难以落地, 大模型智能体可凭借语义理解、知识注入、灵活决策、环境交互方面的优势, 弥补强化学习的不足。因此, 未来基于智能体的渗透测试路线可以融合强化学习优势, 提升学习和决策能力。例如, PTFusion [28] 中基于动态知识图谱适配动态网络场景, AutoPT [20] 采用有限状态机引导动作选择, CHECKMATE [40] 基于经典规实现确定性动作推理, 这些基于智能体渗透测试中的动态增强和推理技术都有望引入强化学习进一步优化。

与基于大模型训练的渗透测试路线融合。虽然大模型的分析和推理能力随着模型的进化在不断增强, 但是依然没有一个能适应所有场景和任务的全能模型。因此, 在基于多智能体的渗透测试架构中, 不同角色的智能体由面向不同任务特性训练或微调的专用模型来承担更合理, 也是未来的发展趋势。例如, ARACNE [36] 采用 GPT-o3-mini、LLaMA 3.1 与 GPT-4o 分别担任规划者、解释者和可选总结者, 实现模块化异构协同。未来面向渗透测试的大模型训练或者微调将不再像 CIPHER [13] 一样针对渗透测试全流程提供辅助, 而是像 RedShell [14] 一样针对多智能体渗透测试框架中的特定角色或任务, 突出聚焦性和专业性。

6 总结

自动化渗透测试是网络安全主动防御的重要技术方向。近年来, 随着大语言模型和智能体技术的快速发展, 自动化渗透测试研究在多智能体协作、领域知识增强、短期记忆管理、推理能力提升、工具调用优化以及测试评估等方面取得了显著进展, 推动该方向由传统规则驱动和单工具调用模式向智能化协同测试模式演进。本文通过对近 3 年 44 项代表性工作的梳理可以看出, 现有研究已在复杂任务分解、知识调用、跨阶段状态管理和多工具协同执行等方面展现出较大潜力。但总体而言, 当前智

能化渗透测试仍处于快速发展阶段，在真实环境适应性、经验复用能力、统一评测基准以及安全可控性等方面仍存在明显不足。未来研究需要进一步加强领域知识与工具知识融合，提升系统在开放环境和未知场景中的泛化能力，构建持续学习与经验沉淀机制，并建立更加统一、可复现、贴近真实应用的评测体系。

致 谢 本工作的完成受到“软件安全智能并行分析”湖南省重点实验室的支持。本文作者诚挚感谢期刊编辑及审稿人为本文审稿及发表所作的贡献。

参考文献

- [1] J. Schwartz and H. Kurniawati, "Autonomous penetration testing using reinforcement learning," arXiv preprint arXiv:1905.05965, 2019.
- [2] J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong, "Autonomous penetration testing using reinforcement learning: A review and perspectives," *Expert Systems with Applications*, p. 130219, 2025.
- [3] S. Zhou, J. Liu, Y. Lu, J. Yang, Y. Zhang, B. Lin, X. Zhong, and S. Hu, "Script: A scalable continual reinforcement learning framework for autonomous penetration testing," *Expert Systems with Applications*, vol. 285, p. 127827, 2025.
- [4] Z. Hu, R. Beuran, and Y. Tan, "Automated penetration testing using deep reinforcement learning," in *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 2–10, IEEE, 2020.
- [5] S. Zhou, J. Liu, D. Hou, X. Zhong, and Y. Zhang, "Autonomous penetration testing based on improved deep q-network," *Applied Sciences*, vol. 11, no. 19, p. 8823, 2021.
- [6] M. C. Ghanem and T. M. Chen, "Reinforcement learning for intelligent penetration testing," in *2018 second world conference on smart trends in systems, security and sustainability (WorldS4)*, pp. 185–192, IEEE, 2018.
- [7] D. López-Montero, J. L. Álvarez-Aldana, A. Morales-Martínez, M. Gil-López, and J. M. A. García, "Reinforcement learning for automated cybersecurity penetration testing," arXiv preprint arXiv:2507.02969, 2025.
- [8] 周仕承, 刘京菊, 钟晓峰, and 卢灿举, "基于深度强化学习的智能化渗透测试路径发现," *计算机科学*, vol. 48, no. 7, pp. 40–46, 2021.
- [9] J. Lan, Z. Lv, J. Li, F. Hao, C. Chen, and Z. Zhang, "A3c reinforcement learning-based automated penetration testing for industrial control systems," in *Third International Conference on Power Electronics and Artificial Intelligence (PEAI 2026)*, vol. 14136, pp. 349–354, SPIE, 2026.
- [10] A. Patki, G. Tayade, P. Jadhav, and A. Kumbhar, "An ai-powered tool for automated web application penetration testing using ppo," *Authorea Preprints*, 2026.
- [11] H. Kong, D. Hu, J. Ge, L. Li, H. Li, and T. Li, "Pentest-r1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning," arXiv preprint arXiv:2508.07382, 2025.
- [12] Y. Yang, L. Chen, S. Liu, L. Wang, H. Fu, X. Liu, and Z. Chen, "Behaviour-diverse automatic penetration testing: a coverage-based deep reinforcement learning approach," *Frontiers of Computer Science*, vol. 19, no. 3, p. 193309, 2025.
- [13] D. Pratama, N. Suryanto, A. A. Adiputra, T.-T.-H. Le, A. Y. Kadiptya, M. Iqbal, and H. Kim, "Cipher: Cybersecurity intelligent penetration-testing helper for ethical researcher," *Sensors*, vol. 24, no. 21, p. 6878, 2024.
- [14] R. Bessa, R. Claro, J. Trindade, and J. Lourenço, "Towards automated pentesting with large language models," arXiv preprint arXiv:2604.11772, 2026.
- [15] T. Y. Zhuo, D. Wang, H. Ding, V. Kumar, and Z. Wang, "Cyber-zero: Training cybersecurity agents without runtime," arXiv preprint arXiv:2508.00910, 2025.
- [16] T. Y. Zhuo, D. Wang, H. Ding, V. Kumar, and Z. Wang, "Training language model agents to find vulnerabilities with ctf-dojo," arXiv preprint arXiv:2508.18370, 2025.
- [17] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "Pentestgpt: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 847–864, 2024.

- [18] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "Pentestagent: Incorporating llm agents to automated penetration testing," in Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (AsiaCCS '25), (Hanoi, Vietnam), pp. 375–391, ACM, 2025.
- [19] L. Muzsai, D. Imolai, and A. Lukács, "Hacksynth: Llm agent and evaluation framework for autonomous penetration testing," arXiv preprint arXiv:2412.01778, 2024.
- [20] B. Wu, G. Chen, K. Chen, et al., "Autopt: How far are we from the end2end automated web penetration testing?," arXiv preprint arXiv:2411.01236, 2024.
- [21] L. Wu, X. Zhong, J. Liu, et al., "Ptgroup: An automated penetration testing framework using llms and multiple prompt chains," in International Conference on Intelligent Computing, pp. 220–232, Springer, 2024.
- [22] H. Kong, D. Hu, J. Ge, et al., "Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework," arXiv preprint arXiv:2501.13411, 2025.
- [23] J. Xu, J. W. Stokes, G. McDonald, et al., "Autoattacker: A large language model guided system to implement automatic cyber-attacks," arXiv preprint arXiv:2403.01038, 2024.
- [24] A. S. Jaswal and A. Baghel, "Awe: Adaptive agents for dynamic web penetration testing," in Proceedings of the Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026), 2026. Co-located with NDSS Symposium 2026.
- [25] M. Vangeli, J. Brynielsson, M. Cohen, and F. Kamrani, "Context relay for long-running penetration-testing agents," in Proceedings of the Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026), 2026. Co-located with NDSS Symposium 2026.
- [26] H. Huang, J. Shi, J. Chen, et al., "Penforge: On-the-fly expert agent construction for automated penetration testing," in Proceedings of the 48th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE'26 NIER), (Rio de Janeiro, Brazil), 2026.
- [27] M. Jiang, J. Huang, J. Wang, Z. Xie, N. Fang, G. Cheng, X. Hu, and H. Wu, "vecho: A paradigm shift from vulnerability verification to proactive discovery with large language models," arXiv preprint arXiv:2603.01154, 2026.
- [28] W. Wang, H. Gu, Z. Wu, H. Chen, X. Chen, and F. Shi, "Ptfusion: Llm-driven context-aware knowledge fusion for web penetration testing," Information Fusion, vol. 127, p. 103731, 2026. Online: September 2025; Print: March 2026.
- [29] J. Zhai, X. Zhou, H. Miao, Z. Li, Z. Li, and H. Yang, "Pentestmcp: Llm and mcp based multi-agent framework for automated penetration testing," Research Square (preprint), 2025. In Review.
- [30] 江颀, 蔡辰旭, 李明达, 陈铁明, 和 朱添田, "基于大语言模型和 rag 的自动化渗透测试框架研究," Telecommunications Science, vol. 41, no. 9, 2025.
- [31] Y. Ginige, A. Niroshan, S. Jain, et al., "Autopentester: An llm agent-based framework for automated pentesting," in 2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp. 163–174, IEEE, 2025.
- [32] J. Henke, "Autopentest: Enhancing vulnerability management with autonomous llm agents," arXiv preprint arXiv:2505.10321, 2025.
- [33] S. Nakatani, "Rapidpen: Fully automated ip-to-shell penetration testing with llm-based agents," arXiv preprint arXiv:2502.16730, 2025.
- [34] I. David and A. Gervais, "Multi-agent penetration testing ai for the web," arXiv preprint arXiv:2508.20816, 2025.

- [35] A. Happe and J. Cito, “Can llms hack enterprise networks? autonomous assumed breach penetration-testing active directory networks,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [36] T. Nieponice, V. Valeros, and S. Garcia, “Aracne: An llm-based autonomous shell pentesting agent,” *arXiv preprint arXiv:2502.18528*, 2025.
- [37] G. Deng, Y. Liu, Y. Li, et al., “What makes a good llm agent for real-world penetration testing?,” *arXiv preprint arXiv:2602.17622*, 2026.
- [38] W. Mai, G. Hong, Q. Liu, et al., “Shell or nothing: Real-world benchmarks and memory-activated agents for automated penetration testing,” *arXiv preprint arXiv:2509.09207*, 2025.
- [39] P. D. Luong, L. T. G. Bao, N. V. K. Tam, et al., “xoffense: An ai-driven autonomous penetration testing framework with offensive knowledge-enhanced llms and multi agent systems,” *arXiv preprint arXiv:2509.13021*, 2025.
- [40] L. Wang, X. Shi, Z. Li, et al., “Automated penetration testing with llm agents and classical planning,” *arXiv preprint arXiv:2512.11143*, 2025.
- [41] B. Challita and P. Parrend, “Redteamllm: an agentic ai framework for offensive security,” *arXiv preprint arXiv:2505.06913*, 2025.
- [42] I. Alshehri, A. Alshehri, A. Almalki, et al., “Breachseek: A multi-agent automated penetration tester,” *arXiv preprint arXiv:2409.03789*, 2024.
- [43] S. G. Bianou and R. G. Batogna, “Pentest-ai, an llm-powered multi-agents framework for penetration testing automation leveraging mitre attack,” in *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, pp. 763–770, IEEE, 2024.
- [44] H. Dai, Y. Li, J. Yan, and Z. Zhang, “Refpentester: A knowledge-informed self-reflective penetration testing framework based on large language models,” in *2025 22nd Annual International Conference on Privacy, Security, and Trust (PST)*, pp. 1–8, IEEE, 2025.
- [45] J. Huang and Q. Zhu, “Penheal: A two-stage llm framework for automated pentesting and optimal remediation,” in *Proceedings of the workshop on autonomous cybersecurity*, pp. 11–22, 2023.
- [46] V. Mayoral-Vilches, L. J. Navarrete-Lozano, M. Sanz-Gómez, L. S. Espejo, M. Crespo-Álvarez, F. Oca-Gonzalez, F. Balassone, A. Glera-Picón, U. Ayucar-Carbajo, J. A. Ruiz-Alcalde, et al., “Cai: An open, bug bounty-ready cybersecurity ai,” *arXiv preprint arXiv:2504.06017*, 2025.
- [47] B. Singer, K. Lucas, L. Adiga, M. Jain, L. Bauer, and V. Sekar, “Incalmo: An autonomous llm-assisted system for red teaming multi-host networks,” *arXiv preprint arXiv:2501.16466*, 2025.
- [48] T. Abramovich, M. Udeshi, M. Shao, K. Lieret, H. Xi, K. Milner, S. Jancheska, J. Yang, C. E. Jimenez, F. Khorrami, et al., “Enigma: Enhanced interactive generative model agent for ctf challenges,” *arXiv preprint arXiv:2409.16165*, 2024.
- [49] K. Nakano, R. Fayyazi, S. J. Yang, and M. Zuzak, “Guided reasoning in llm-driven penetration testing using structured attack trees,” *arXiv preprint arXiv:2509.07939*, 2025.
- [50] P. He, A. Fox, L. Miculicich, S. Friedli, D. Fabian, B. Gokturk, J. Tang, C.-Y. Lee, T. Pfister, and L. T. Le, “Co-redteam: Orchestrated security discovery and exploitation with llm agents,” *arXiv preprint arXiv:2602.02164*, 2026.
- [51] S. Caldwell, M. Harley, M. Kouremetis, V. Abruzzo, and W. Pearce, “Pentestjudge: Judging agent behavior against operational requirements,” *arXiv preprint arXiv:2508.02921*, 2025.
- [52] R. Yang, M. Cheng, G. Deng, T. Zhang, J. Wang, and X. Xie, “Pentesteval:

Benchmarking llm-based penetration testing with modular and stage-level design,” arXiv preprint arXiv:2512.14233, 2025.

- [53] L. Gioacchini, A. Delsanto, I. Drago, M. Mellia, G. Siracusano, and R. Bifulco, “Autopenbench: A vulnerability testing benchmark for generative agents,” in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 1615–1624, 2025.
- [54] X. Shen, W. Cheng, Y. Chen, Z. Li, Y. Gu, L. Wang, W. Zhao, D. Sun, and J. Wang, “Aeas: Actionable exploit assessment system,” arXiv preprint arXiv:2509.17832, 2025.
- [55] I. Isozaki, M. Shrestha, R. Console, and E. Kim, “Towards automated penetration testing: Introducing llm benchmark, analysis, and improvements,” in Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization, pp. 404–419, 2025.
- [56] Y. Zhu, A. Kellermann, D. Bowman, P. Li, A. Gupta, A. Danda, R. Fang, C. Jensen, E. Ihli, J. Benn, et al., “Cve-bench: a benchmark for ai agents’ ability to exploit real-world web application vulnerabilities,” arXiv preprint arXiv:2503.17332, 2025.
- [57] Z. Luo, P. Wang, B. Wang, Y. Tang, W. Xie, X. Zhou, D. Liu, and K. Lu, “Vulhawk: Cross-architecture vulnerability detection with entropy-based binary code search,” in NDSS, 2023.
- [58] X. Zhou, P. Wang, C. Liu, T. Yue, Y. Liu, C. Song, K. Lu, Q. Yin, and X. Han, “Ultrafuzz: Towards resource-saving in distributed fuzzing,” IEEE Transactions on Software Engineering, vol. 49, no. 4, pp. 2394–2412, 2022.
- [59] D. Liu, P. Wang, X. Zhou, W. Xie, G. Zhang, Z. Luo, T. Yue, and B. Wang, “From release to rebirth: Exploiting thanos objects in linux kernel,” IEEE Transactions on Information Forensics and Security, vol. 18, pp. 533–548, 2022.



Wang Peng-Fei

Born in 1988.09, Ph.D, Associate Professor, major in software vulnerability mining, penetration testing.



Zhang Wen-Hua

Born in 2002.06, Master student, major in penetration testing.



Wang En-Ze

Born in 1996.04, Ph.D, Assistant Professor, major in software vulnerability mining, penetration testing.



Zhou Yuan

Born in 1998.07, Ph.D student, major in Intelligent Cybersecurity, penetration testing.



Chen An-Ying

Born in 1996.11, M.S. Her research interests include software and system security, AI agent for security.



Xia Tian

Born in 2001.03, Ph.D student, major in Intelligent Cybersecurity penetration testing.



Tao Yi

Born in 2004.10, undergraduate student, major in Cyberspace Security.



Xie Wei

Professor, PhD Supervisor. Research
Interests: Intelligent Cybersecurity,
Security of Intelligent Systems.

discussion from isolated system demonstrations toward a clearer understanding of the overall research landscape and its future direction.

Background This survey focuses on the problem of LLM-driven autonomous penetration testing. More specifically, it addresses the challenge of enabling large language models (LLMs) to perform or assist with long-horizon, multi-step offensive security tasks, such as vulnerability discovery, exploitation planning, privilege escalation, lateral movement, and attack-chain generation. Unlike conventional penetration testing tools that rely heavily on predefined rules, signatures, or expert-crafted workflows, this problem requires an intelligent system to reason across complex environments, integrate external tools, adapt to intermediate observations, and maintain coherence over extended task horizons.

Internationally, this problem has attracted growing attention with the rapid development of foundation models and agent systems. Existing studies have shown that LLMs can already provide useful support in several penetration-testing subtasks, including vulnerability explanation, exploit generation, command recommendation, report drafting, and attack-path planning. More advanced systems, such as PentestGPT, VulBot, and related agent-based security frameworks, have further demonstrated the feasibility of combining LLM reasoning with external tools and structured environments to execute more realistic offensive workflows. However, the current state of the art is still far from fully autonomous and reliable penetration testing. Major limitations remain in long-term planning, state tracking, tool orchestration, environment grounding, error recovery, and the safe execution of high-risk actions. In addition, current evaluation settings are often simplified, making it difficult to determine how well these systems generalize to realistic enterprise environments.

This survey contributes to the field by providing a systematic understanding and structured synthesis. Rather than proposing a new penetration-testing agent, it organizes recent international research into a coherent analytical framework. Specifically, it reviews representative LLM-based pentesting systems, summarizes their task decomposition strategies, memory and planning mechanisms, tool-use paradigms, and evaluation methodologies, and analyzes their common strengths and bottlenecks. On this basis, the survey clarifies the current technical boundaries of LLM-powered pentesting and identifies key open challenges for future research. Therefore, this work contributes by moving the